Rowhammer on Raspberry Pi's

A Proof of Concept Rowhammer Attack to analyze the susceptibility of current Raspberry Pi models

P r a c t i c a l W o r k

University of Applied Science Hof
Faculty Computer Science
Degree Program Medieninformatik

Submitted to
Prof. Dr. Florian Adamsky
Alfons-Goppel-Platz 1
95028 Hof

Submitted by
Jakob Drescher
Mtr. Nr.: 00498922
Steinäcker 10
95191 Leupoldsgrün

Hof, February 24, 2026

Abstract

Single-board computers such as Raspberry Pi 4 and 5, represent compact and power-efficient systems intended for diverse areas of applications. These platforms depend on main memory to supply fast and temporary storage for data currently being processed. In Dynamic Random-Access Memory (DRAM), each bit is maintained within an individual memory cell, where a binary state is encoded by the existence or absence of electrical charge in a capacitor [87, 30, 79]. As DRAM cells are further miniaturized to achieve higher memory densities within constrained physical space, advances in technology improve storage capacity and energy efficiency but also increase vulnerability to disturbance phenomena. Reliable DRAM requires maintaining data integrity despite charge leakage. Repeated gate-induced drain- and subthreshold leakage [87, 26] can accelerate charge loss and intensify cell-to-cell interference, eventually causing bits to flip from 0 to 1 or vice versa. Rowhammer leverages this behavior by intentionally provoking rapid, repeated row activations that intensify charge loss and disturbance errors. As a result, memory contents may be corrupted, potentially facilitating privilege escalation and other critical security impacts. In this work, we develop a Rowhammer proof-of-concept for the Raspberry Pi 4 and 5. The implementation enables direct DRAM access by employing cache-bypass techniques based on eviction strategies and ARM cache maintenance instructions. To facilitate targeted row accesses, DRAM address mapping was reconstructed by ported ARM-compatible DRAMA tool. Additionally, TRRespass was adapted to ARM to evaluate effective access patterns for hammering designed to induce bitflips. The proof-of-concept integrates all functional components, ranging from cache-bypass mechanisms and targeted DRAM access to effective hammering patterns, into a framework designed to experimentally assess susceptibility to Rowhammer on contemporary Raspberry Pi hardware. Under the tested conditions, no susceptibility to Rowhammer was observed on modern Raspberry Pi models, regardless of model or RAM configuration. Hardware protections and DRAM page policies limit the effectiveness of hammering patterns, demonstrating the practical challenges in exploiting Rowhammer on these platforms.

Keywords*: Rowhammer, DRAM, Raspberry Pi, Address Mapping, Target Row Refresh (TRR)*

Contents

List of Figures

List of Abbreviations

AOSP	Android Open Source Project	15
CPU	Central Processing Unit	3, 5, 10, 11, 13, 14, 15, 16, 19, 22, 30, 33
DDR	Double Data Rate	39
DMA	Direct Memory Access	15
DRAM	Dynamic Random-Access Memory	II, 1, 2, 3, 9, 10, 13, 14, 15, 20, 22, 25, 26, 29, 32, 33, 34, 35, 36, 37, 38, 39, 40, 42
DSB	Data Synchronization Barrier	23
DVFS	Dynamic Voltage and Frequency Scaling	11
DZE	Data Zeroing Extension	23, 24
GPU	Graphics Processing Unit	15, 16
ISB	Instruction Synchronization Barrier	23
LRU	Least Recently Used	17
MAC	Maximum Activate Count	38
MMU	Memory Management Unit	3, 35
PIPT	Physically Indexed, Physically Tagged	6, 17
PIVT	Physically Indexed, Virtually Tagged	6
PMCCNTR	Performance Monitoring Cycle Count Register	31
PMU	Performance Monitoring Unit	4, 23, 27
PoC	Point of Coherency	21
PTE	Page Table Entry	3
RAM	Random-Access Memory	3
RDTSC	Read Time-Stamp Counter	4
SoC	System-on-Chip	10
SPD	Serial Presence Detect	38, 39
SSH	Secure Shell	12
TLB	Translation Lookaside Buffer	3, 6
TMUX	Terminal Multiplexer	12
TRR	Target Row Refresh	II, 9, 13, 37, 38, 39, 40, 42
UCI	Unprivileged Cache Invalidation	IX, 23, 24
VIPT	Virtually Indexed, Physically Tagged	6, 17

VIVT	Virtually Indexed, Virtually Tagged	5
WC	Write-Combining	14

Listings

List of Tables

1 Introduction

DRAM is the dominant main memory technology used in contemporary computing systems [7]. DRAM stores each bit of information in a memory cell that consists of a capacitor and an access transistor. Due to charge leakage in the capacitor, periodic refresh operations are required to maintain data integrity. Otherwise, the stored data may be lost over time [89]. The need to accommodate ever-larger memory capacities in constrained physical space has led to a continuous reduction in DRAM cell dimensions and a corresponding increase in memory density. Consequently, memory cells can function with lower operating voltages and smaller charge levels, enabling higher storage capacity and improved energy efficiency [25]. Unfortunately, this scaling led to a problem known as disturbance error. A hardware-induced failure that arises from cell-to-cell interference in scaled memory technologies [64], leading in bitflips.
Kim et al. [87] exploited this error in 2014 as an attack, also known as Rowhammer. Over the years, this attack was further developed, first on FPGA-based memory controller, then x86 followed involuntarily, and later the attack also reached ARM systems [30]. Primarily on mobile phones and chromebooks [79, 68, 65, 66]. Ultimately, it also found its way onto embedded systems and single-board computers. In recent years, analyses have been carried out on some older Raspberry Pi models. However, due to power and die area restrictions in LPDDR4 [65] as well as temperature controlled refresh rate [50, 51, 41, 42], this attack remains relevant for contemporary Raspberry Pi models.
This study focuses on the implementation of a Rowhammer-based proof-of-concept. This proof-of-concept is used to demonstrate the occurrence of bitflips on modern Raspberry Pi platforms, specifically the Raspberry Pi 4B and Raspberry Pi 5.

Outline. The remainder of this study is structured as follows. Section 2 "*Background*" provides an overview of the technical foundations required for this work. Section 3 "*Methodology*" describes the practical approach used to design, implement, and evaluate the proof-of-concept with details on the experimental setup of the test systems and environment. Section 4 "*Overview of Challenges*" introduces the fundamental techniques required to construct the attack, followed by section 5 "*Direct DRAM Activation*", section 6 "*Address Mapping Reconstruction*", and section 7 "*Outpacing DRAM refresh*" that describe the individual challenges in terms of implementation, design choices, and practical constraints. Section 8 "*Conclusion*" synthesizes the key challenges and insights, and underscores the main findings of this work.

2 Background

2.1 Rowhammer Attack

Rowhammer is a main memory attack triggered by a software-induced hardware fault to flip bits soley with increased pressure caused by frequent reads to selected memory rows.
An overview of the Rowhammer attack workflow is provided in fig. 1. From a technical perspective, the Rowhammer workflow begins with the attacker placing data in memory through arbitrary memory allocation. Subsequently, specific DRAM rows are filtered and addressed together in a targeted manner by reading them at a very high frequency, a process commonly referred to as “hammering”. During this process, the cache is consistently bypassed using eviction strategies or explicit flush instructions. The hammering technique is crucial, as it determines which victim row is targeted, how many aggressor rows are involved, and their relative distances to the victim rows. These factors decisively influence the frequency of observable bitflips, even in the presence of mitigation mechanisms.
As a consequence of continuous hammering, adjacent DRAM rows are exposed to repeated gate-induced drain and subthreshold leakage [87, 26]. This leads to sufficient charge loss [89] and electrical disturbance [44], ultimately causing memory cells to flip from 0 to 1 or vice versa.

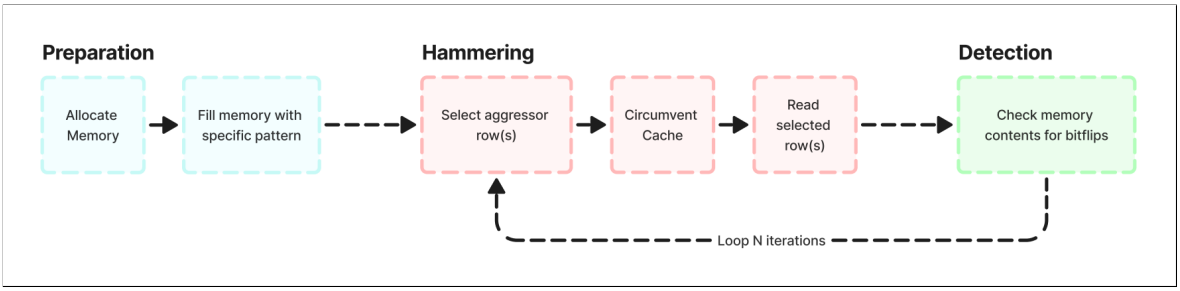

Figure 1: Overview of the Rowhammer Attack Procedure

2.2 Memory Addressing

Building on the preceding explanation of how Rowhammer is executed, it is essential to examine how software running on a system is able to access hardware components such as DRAM. This interaction fundamentally relies on the use of memory addresses. It requires memory addresses as they uniquely identify and locate data, allowing systems to store and later retrieve the same information reliably, independent of its physical realization. In this context, two types of memory addresses can be distinguished. Virtual addresses are generated and used by processes and define a per-process virtual address space that is independent of the

underlying physical memory [63]. The size of this virtual address space depends on the system architecture, such as the Central Processing Unit (CPU) register width, and may exceed the amount of available physical Random-Access Memory (RAM) [84]. In contrast, physical addresses refer to actual locations in main memory and constitute the system-wide physical address space corresponding to the installed RAM [61]. Physical addresses are globally unique within the system [84]. Processors employ virtual memory systems that translate virtual addresses into physical addresses [39]. The system achieves these memory translations, also referred to as mappings, by resolving the physical memory address corresponding to a given virtual address [61]. Page tables play a central role in defining and managing these virtual-to-physical address mappings. A page table consists of numerous Page Table Entry (PTE)s [61]. Page tables store the mapping between virtual and physical addresses, including associated attributes. The Translation Lookaside Buffer (TLB) uses these entries as a fast cache to accelerate address translation [84, 75]. The kernel provides an interface exposed via `/proc/pagemap`, which returns a 64-bit value containing page table-related information. This information includes whether and how a page is located in RAM or swap, as well as which physical memory area it maps to [74]. Page tables are typically managed by the Memory Management Unit (MMU) [62, 61]. The MMU is a hardware component responsible for translating virtual addresses into physical addresses [61]. When the CPU attempts to access a memory location, it supplies a virtual address to the MMU, which then verifies if a corresponding translation already exists in the TLB [75]. On a memory access, the CPU supplies a virtual address to the MMU, which first consults the TLB, a cache of recent address translations integrated into the MMU [62], to quickly resolve the corresponding physical address [75]. By avoiding frequent page table lookups in main memory, the TLB significantly reduces the latency of address translation [62, 61]. Functionally, the virtual address is first divided into a virtual page number and a page offset [61]. The page offset is passed through unchanged, as it does not require translation. The virtual page number is then looked up in the TLB by searching for a matching tag. In the case of a TLB hit, it returns the physical address immediately. In the case of a TLB miss, the MMU accesses main memory to locate the corresponding page frame. If the entry is found, the translation is completed and cached in the TLB. If no entry exists, the system may need to retrieve the page from storage, which represents the worst-case scenario [61].

2.3 Timing Measurement

Timing serves as the basic for identifying DRAM accesses and is an essential part of executing or reverse engineering rowhammer attacks, particularly when adapting such attacks to ARM-

based systems for the development of proof-of-concept implementations. On x86 architectures, high-resolution timestamps can be obtained in an unprivileged context using the Read Time-Stamp Counter (RDTSC) instruction [10, 48, 22, 35]. The RDTSC instruction provides an unprivileged mechanism for acquiring sub-nanosecond timing information. It functions as a fine-grained timer by returning a high-resolution count of CPU clock cycles [29]. In contrast, ARM architectures, specifically ARMv7-A and ARMv8-A, do not offer a direct counterpart to the RDTSC instruction. Instead, high-resolution timing on these systems can be acquired via the Performance Monitoring Unit (PMU) cycle counter, referred to as `PMCCNTR`, which counts processor cycles. While these measurements enable fast and precise timing, access to the performance counters is generally restricted to kernel space by default [57, 50].
The `PMCCNTR_EL0` register holds the value of the processor cycle counter, monitoring the number of processor clock cycles [20]. It is a 64-bit register integrated within the PMU block. Architecturally, the external register `PMCCNTR_EL0` bits `[63:0]` map to the AArch64 system register. The `PMCR_EL0 LC, D` control bits determine the increment behavior of the cycle counter, allowing it to advance either on every processor clock cycle or on every 64th processor clock cycle. Following a warm reset, `PMCCNTR_EL0` is initialized to an architecturally undefined value [20].

2.4 Cache Organization

In a direct-mapped cache, each main memory entry map to exactly one cache location. As a consequence, multiple distinct memory addresses can also map to the same cache location [57].
As seen in the example of fig. 2, there is a 4 KB direct-mapped cache with 128 cache lines of 64 bytes each, where addresses are split into a 6-bit block offset, 7-bit index, and 19-bit tag. To determine whether a specific address is present in the cache, the index bits of the address are first identified, and the associated cache line's stored tag is then compared with the address's tag. If the tags match and the valid bit is set, a cache hit occurs, and the block offset is used to select the relevant word from the cache line. If the tags do not match or the valid bit is not set, indicating that the cache entry does not contain valid data, a cache miss occurs, and the cache line is replaced with data fetched from the requested memory address. If the replacement policy can choose any entry in the cache to hold the data, the cache is referred to as fully associative [57]. Nevertheless, modern processors typically employ set-associative caches, which organize cache lines into multiple sets [57]. In an N-way set-associative cache, each memory block can be placed in any one of N cache lines within a specific set.

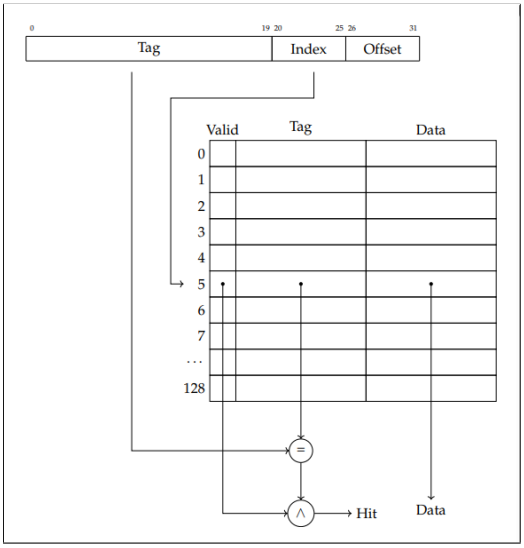

Figure 2: Direct-mapped cache [57]

Each memory address maps to exactly one cache set, and addresses that map to the same set are called congruent addresses, as illustrated in fig. 3. When all cache lines of a set are occupied and a new block needs to be loaded, the cache replacement policy determines which existing cache line is evicted in order to accommodate the new data [57].

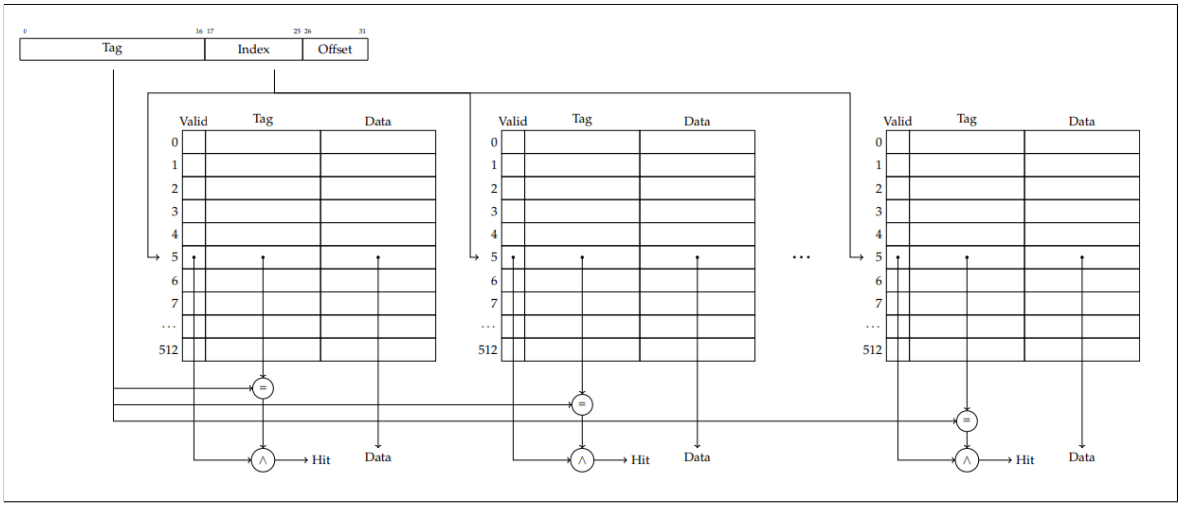

Figure 3: N-way associative cache [57]

CPU caches can be indexed and tagged using either virtual or physical addresses. When using virtual addresses, it is possible for the same physical address to be cached in multiple cache lines, which can negatively affect performance. Cache lines are uniquely identified by a tag, which may be derived from either the virtual or the physical address [57].
In **Virtually Indexed, Virtually Tagged (VIVT)** caches, both the index and the tag are derived from the virtual address. Such caches are generally faster because no address translation is

required before lookup. Yet, the virtual tag is not unique, and shared memory may appear multiple times in the cache.

Physically Indexed, Physically Tagged (PIPT) caches use the physical address for both index and tag. This approach avoids duplication of shared memory in the cache, but it is slower because address translation via the TLB is required before cache access.

Physically Indexed, Virtually Tagged (PIVT) caches use the physical address for indexing and the virtual address for tagging. This design provides no practical benefit, as it still requires address translation while retaining the disadvantage of non-unique virtual tags, allowing shared memory to be duplicated.

Virtually Indexed, Physically Tagged (VIPT) caches use the virtual address for indexing and the physical address for tagging. This approach offers lower latency than PIPT caches because index lookup can proceed in parallel with TLB translation, although tag comparison must still wait for the physical address.

3 Methodology

This section presents the methodological framework, focusing on the design and implementation of the Rowhammer Proof of Concept. It summarizes the overall research design on selected devices and specifies the requirements that guided the implementation of the experimental setup

3.1 Research Design

The research conducted in this thesis follows a empirical and quantitative study based on controlled experiments designed to evaluate the vulnerability of different Raspberry Pi models to Rowhammer.
The study primarily employs a deductive methodology, testing established Rowhammer theories on Raspberry Pi platforms. It builds on existing theoretical foundations, formulates hypotheses and subsequently validates or refutes them through targeted experiments. Complementing this, an inductive component is incorporated, as experimental observations may reveal unanticipated behaviors, from which new hypotheses or insights can be derived. Overall, the research is predominantly deductive, with supporting inductive elements informed by empirical findings.
The experimental evaluation is conducted on multiple Raspberry Pi devices, see table 1. The tested hardware includes Raspberry Pi 4 models with 4 GB and 8 GB of RAM, as well as Raspberry Pi 5 models with 4 GB, 8 GB, and 16 GB of RAM.

Table 1: Overview of the Test Devices and Memory Configurations

Platform	**Processor**	**Architecture**	**SoC**	**DRAM**
Raspberry Pi 4B	ARMv8	aarch64	Broadcom BCM2711	4 GB LPDDR4
Raspberry Pi 4B	ARMv8	aarch64	Broadcom BCM2711	8 GB LPDDR4
Raspberry Pi 5	ARMv8	aarch64	Broadcom BCM2712	4 GB LPDDR4x
Raspberry Pi 5	ARMv8	aarch64	Broadcom BCM2712	8 GB LPDDR4x
Raspberry Pi 5	ARMv8	aarch64	Broadcom BCM2712	16 GB LPDDR4x

The Raspberry Pi 4 is based on a quad-core ARMv8 Cortex-A72 CPU clocked at 1.8 GHz. Each core is equipped with an L1 cache consisting of 32 KB data cache and 48 KB instruction cache, and all cores share a 1 MB L2 cache. This configuration makes the Raspberry Pi 4 approximately 50 % faster than the Raspberry Pi 3B [71].
The Raspberry Pi 5 features a quad-core ARMv8 Cortex-A76 CPU operating at 2.4 GHz. Each core provides a 64 KB L1 cache for both data and instructions, a dedicated 512 KB L2 cache, and a shared 2 MB L3 cache. In aggregate, the architectural improvements introduced with

the BCM2712 result in a performance uplift of approximately two to three times compared to the Raspberry Pi 4 for common CPU- or I/O-intensive workloads [71].
The selected ARM-based single-board computers represent current and widely used generations of low-cost, making them suitable targets for evaluating the susceptibility to Rowhammer attacks across different hardware generations within a consistent platform family. The hardware shares similar architectural features and system layouts, allowing the proof-of-concept experiments to systematically compare Rowhammer effects across different models.
The study primarily draws upon technical documentation, datasheets, and relevant academic literature forming the primary foundation for the study's analysis, complemented by GitHub repositories on Rowhammer experiments to understand and adapt methodologies for the target architecture
We collect data through monitoring and documenting system responses after observations of system behavior while executing Rowhammer experiments on both Raspberry generations.
The data analysis involved the development and application of scripts to process experimental outputs, complemented by visualizations that enabled direct comparison across devices. Multiple logging formats, including `.log`, `.dat`, and `JSON`, were employed to ensure comprehensive data capture and support reproducibility. The resulting findings were interpreted within the framework of the experimental design to assess device susceptibility.

3.2 Evaluation

The evaluation involved repeated execution of experiments under identical conditions to ensure the stability and consistency of observed results, conducted within a controlled environment to minimize external interference, and incorporated statistical analyses to assess the significance, reliability, and comparability of the measured outcomes.
The experiments were carried out on the provided devices within a secured setup to prevent any accidental harm. This study emphasizes the responsible and academic investigation of Rowhammer vulnerabilities, with results intended solely for analysis and evaluation, without posing any risk to other systems.

3.3 Requirements

To ensure a structured and reliable experimental evaluation of Rowhammer attacks on Raspberry Pi platforms, a set of technical, and ethical requirements is defined. These requirements guide the design, implementation, execution, and evaluation of the experimental framework and ensure that the results are reproducible, and meaningful.

3.3.1 Functional requirements

The developed system must support the usage of different hammering techniques, including double-sided, many-sided, and one-location approaches. Furthermore, for precise control over the hammering process, the configuration of the number of iterations and hammer cycles must be adjustable. In addition, the system must automatically detect the pagesize of the underlying environment to ensure correct memory allocation and mapping behavior. An input mechanism for bank addressing functions must be provided. This enables the integration of experimentally derived DRAM addressing schemes into the hammering process. To guarantee direct DRAM access patterns, the system must implement cache bypass mechanisms. Additionally, it must support virtual-to-physical-to-DRAM mapping to accurately translate memory addresses across the entire address translation hierarchy. The system must further incorporate mitigation bypass capabilities, such as the circumvention of TRR mechanisms. Hammering must be combined with bitflip detection, including for example, transitions from $0 \rightarrow 1$ and from $1 \rightarrow 0$, in order to reliably identify memory disturbances. Finally, the system must provide comprehensive logging capabilities, supporting output in terminal format as well as structured formats such as JSON, CSV, and `.log` files.

3.3.2 Non-functional requirements

The implementation requires root privileges in user space to execute privileged instructions. All Raspberry Pi 4 and Raspberry Pi 5 devices must operate in ARMv8 (AArch64) mode. The system must meet requirements regarding performance and efficiency and include robust error-handling mechanisms. Reproducibility is essential, such that repeated measurements yield similar results under comparable conditions.

3.4 Lab Setup

The laboratory setup was designed by us to establish a consistent and reproducible environment for both system configuration and experimental execution, ensuring all devices, software, and development workflows were properly prepared prior to conducting experiments.

3.4.1 System Setup

For the Raspberry Pi systems, we generated public and private keys for WireGuard and SSH, distributed the public keys to enable secure access, and verified connectivity on all devices. Direct access from the laptop was established by configuring authorized keys on each Pi and updating the laptop's `/etc/hosts` with all relevant IP addresses.

After completing the system setup, we prepared multiple Raspberry Pi platforms for experimentation. All platforms were based on the ARMv8 architecture, providing a common architectural baseline while differing in memory capacity and System-on-Chip (SoC) generation. At the outset of the project, the test systems did not contain the previously referenced data, differed in processor and architecture, and were based on different operating systems and kernel versions, leaving the Raspberry Pi devices misaligned and inconsistent with one another. These discrepancies affected the consistency of the experimental setup.

On the Raspberry Pi 4 devices, older system installations were used. These systems ran Arch Linux ARM and reported a CPU identified as an ARMv7 Processor revision 3 (v7l), which were limited to 32-bit execution [72]. The kernel versions differed between these devices, with version `6.12.25-1-rpi` on the 4 GB model and version `6.12.21-1-rpi` on the 8 GB model. Raspberry Pi 4 devices can run kernel modules, but the armv7l architecture restricts user-mode execution from performing cache maintenance instructions like `DC CICAC`, which are limited to privileged modes.

The Raspberry Pi 5 devices also initially ran older system installations based on Arch Linux ARM. These systems reported an ARMv8 Processor revision 1 (v8l) and used the aarch64 architecture, which supports both 32-bit and 64-bit computation [89]. The Raspberry Pi 5 ran the kernel version `6.12.47.1-rpi-16k` but mismatched headers between kernel and header prevented successful compilation and insertion of kernel modules. After performing a full system upgrade using the command `pacman -Syu linux-rpi-16k linux-rpi-16k-headers`, the kernel was updated, but no corresponding newer header package was available. As a result, the kernel and header versions remained different. Attempts to manually downgrade the kernel were also unsuccessful. Installing a cached kernel package (`linux-rpi-16k-6.12.46-1`) via `pacman -U` and rebooting did not result in the expected kernel version, even though the package was present on the system. Other kernel versions could be installed successfully, but still without matching header versions. Furthermore, a repository inconsistency was observed. Although the kernel appeared outdated on the systems, the kernel and headers shared the same version in the repositories. This discrepancy may have been caused by mirrors lagging behind the Arch Linux ARM website. After all attempted approaches failed, a final decision was made to align the systems by reinstalling all Raspberry Pi devices. The reinstall was motivated by several factors. First, the inability to execute kernel modules reliably hindered further experimentation. Second, the absence of cache flush instructions on ARMv7 systems prevented the required cache management functionality. Reinstalling the systems would provide deterministic cache-line invalidation, ensuring that every subsequent memory access is served from DRAM. Finally, a clean reinstallation reduced setup complexity by avoiding the need for eviction-set construction and timing-based

profiling, thereby establishing a consistent and reliable experimental environment across all platforms.

We reinstalled the Raspberry Pi systems using the Raspberry Imager, configuring the hostname, user credentials, LAN-only connectivity, timezone, keyboard layout, and SSH with pre-existing public keys. After flashing and automatic boot, we verified connectivity. We updated and upgraded the systems, and unified shell configurations. We installed and verified WireGuard VPN, enforced internet connectivity over HTTPS, and installed essential development tools. We cloned our GitHub repository [38] for the developing the proof-of-concept and configured Git with the appropriate user credentials. For Python development, a virtual environment was created, and all additional dependencies were installed.

Since Hugepages were not enabled on this kernel configuration, we applied the necessary modifications to activate Hugepage support by rebuilding the kernel natively on the target systems. The procedure began with installing necessary build dependencies. These included standard compilation tools as well as libraries required for building the Linux kernel and device tree binaries for the Raspberry Pi platform. The official Raspberry Pi Linux kernel source code [70] was then cloned, restricted to the latest commit of `rpi-6.12.y`.

We reused the active kernel configuration as a baseline, modifying options to enable HugeTLB and Transparent Hugepage support. We then compiled the kernel, modules, and device tree using all available cores, installed the modules, and deployed the new kernel and device tree binaries to the boot directory. After reboot, we verified the running kernel version and confirmed that both HugeTLB and Transparent Hugepages were correctly enabled and recognized by the operating system.

Dynamic Voltage and Frequency Scaling (DVFS) adjusts CPU frequency and voltage according to workload, temperature, and energy constraints, reducing frequency under low load and increasing it under high load. In our experiments, active DVFS introduces measurement inaccuracies due to fluctuating CPU frequencies the CPU is not constant and distort threshold and timings of cycle measurements. This behavior is unsuitable for experiments requiring precise and stable latency measurements. System inspection showed that the default `ondemand` governor dynamically scales frequencies within the supported range of 600 MHz to 1.5 GHz, with observable fluctuations during individual measurement runs, confirming active DVFS. After switching the governor to `performance`, frequency scaling was disabled, resulting in substantially more accurate and consistent measurements. All newly deployed systems are based on Raspberry Pi OS Lite 64-bit running on the aarch64 architecture. This paper primarily focuses on Rowhammer techniques targeting the ARMv8-A architecture. The Raspberry Pi 4 uses kernel version `6.12.66-v8+` and `6.12.67-v8+`, respectively. While the Raspberry Pi 5 runs kernel version `6.12.67-v8-16k+`. As a result of this setup, all test

systems now operate on the same operating system, have access to all maintenance instructions that are relevant in section 5.4, are capable of executing kernel modules and provide HugePage support enabled through a natively rebuilt kernel. This unified system configuration provides several additional advantages. It ensures improved comparability of experimental results, as all Raspberry Pi devices run on an identical software stack.

3.4.2 Experiment Setup

For long-term experiments, the Terminal Multiplexer (TMUX) tool was installed to address the challenge of maintaining continuous processes on the Raspberry Pi systems while allowing the user to disconnect from the Secure Shell (SSH) session. TMUX is a terminal multiplexer that enables multiple terminal sessions to be created, accessed, and controlled independently within a single SSH connection. New sessions can be initiated, allowing experiments to be executed within an isolated session by continuously running the experiment script even if the SSH connection was closed. Detached sessions could later be reattached, and sessions that were no longer needed could be terminated. This setup ensured reliable and uninterrupted execution of long-duration experiments while maintaining full control over multiple concurrent tasks.

3.4.3 Automated Scripts

Automated scripts were developed to streamline experiment management. They check host reachability, verify connectivity, and manage terminal sessions. Some scripts open terminals and initiate SSH connections to reachable hosts, while others create, list, attach, or terminate TMUX sessions, skipping hosts that are unreachable or already running the relevant session. Session summaries, counts, and status indications are provided automatically.

4 Overview of Challenges

For Rowhammer attacks to be feasible, three fundamental prerequisites must be satisfied, commonly referred to as attack primitive [23, 31]. These primitives define the necessary properties of memory accesses that

C1. Direct DRAM Activation In section 5, we discuss the first prerequisite, which requires that memory accesses bypass the CPU cache and reach the DRAM directly. DRAM cells are only electrically stressed when the memory controller opens and closes the corresponding DRAM row, which occurs exclusively on accesses that are not served by the cache. Ensuring that each memory access physically toggles DRAM cells, rather than being handled by the CPU cache, is therefore essential. This condition is critical because Rowhammer relies on high-frequency direct DRAM activations, which can only be achieved when cached accesses are avoided. If memory accesses remain cached, the hammering speed is reduced by orders of magnitude and falls far below the threshold necessary to discharge adjacent cells. Consequently, forcing cache misses or using uncached memory regions is mandatory to generate the required activation frequency.

C2. Address Mapping Reconstruction In section 6, we specify the second prerequisite, where memory accesses must be directed to precisely chosen physical DRAM rows. To accomplish this, an attacker must determine how virtual addresses are mapped onto the underlying physical DRAM structures in order to identify aggressor rows that surround a specific victim row. This process requires reconstructing the DRAM address mapping scheme. Only with accurate knowledge of this mapping can hammering be focused such that specific adjacent rows are repeatedly activated. Without correct targeting, hammering becomes largely probabilistic and rarely, if ever, results in disturbances of victim data.

C3. Outpacing DRAM refresh Finally in section 7, we demand that memory accesses occur at a rate that outpaces the DRAM refresh mechanism. The attack fundamentally depends on hammering aggressor rows faster than refresh cycles can restore the electrical charge in adjacent victim cells. Achieving this requires a high access throughput to the same DRAM rows with minimal delay between consecutive activations. In practice, this also implies that mitigation mechanisms designed to limit activation frequency or to increase refresh activity - such as TRR or an increased refresh rate - often need to be bypassed. Without overcoming such defenses, the hammering intensity remains insufficient to reliably induce bitflips.

5 Direct DRAM Activation

To achieve "*C1. Direct DRAM Activation*", the cache must be completely bypassed so that all memory accesses repeatedly reach the DRAM without being served from any cache level. This section therefore provides a brief discussion of the available methods to bypass CPU caches, explains why certain approaches are selected or discarded, and identifies the most effective method on ARM-based systems such as the Raspberry Pi. In addition, it outlines the general implementation process, including the main development steps and the challenges encountered during implementation.
From a high-level perspective, several methods are considered and evaluated as potential approaches to achieve uncached memory accesses. These methods include non-temporal accesses, explicitly uncached memory, cache eviction, and cache flushing. Each approach differs in its feasibility, effectiveness, and platform dependency, particularly on ARM-based architectures.

5.1 Non-temporal accesses

Non-temporal accesses have been proposed as a cache-bypassing technique in the context of Rowhammer attacks. In particular, [73] introduce a Rowhammer approach based on x86 non-temporal instructions. Non-temporal data refers to data that is accessed once and not reused in the near future, exhibiting low temporal locality, such as streaming data, which ideally should not be cached in order to avoid cache pollution [89]. This cache-bypassing capability facilitates Rowhammer attacks on x86 architectures, as demonstrated in [73]. However, as mentioned in [79, 89, 11], ARMv8-A non-temporal memory access instructions merely provide a hint to the memory system that caching is not required, and in practice the accessed data is still cached. Although the ARMv8-A instruction set includes non-temporal load and store instructions, such as `LDNP` and `STNP`, it is generally considered that these instructions are not effective for triggering the RH bug [89]. This observation is further reinforced by the statement in [79] that these ARMv8-A non-temporal memory access instructions do not reliably bypass the cache and that data accessed through them is still found cached in practice [89]. Conceptually, non-temporal operations do not follow standard cache-coherency rules and are assumed by the memory controller to access data that will not be reused, and several non-temporal instructions are designed to bypass the cache. Nevertheless, a major limitation of this approach is that all non-temporal stores to a single address are combined into a single Write-Combining (WC) buffer, such that only the last write is eventually forwarded to DRAM, regardless of how many stores are issued. As a result, the effective access rate

is insufficient to perform reliable hammering [31]. Since Rowhammer requires a very high access rate, this limitation makes non-temporal accesses unsuitable for this purpose [39].

5.2 Uncached memory

Another approach is the use of explicitly uncached memory. This method allocates memory regions that are explicitly marked as uncached, for example via Android ION, such that every memory access goes directly to DRAM without interacting with the CPU cache. This approach is particularly well suited for mobile ARM platforms, such as ARMv7 and ARMv8 systems, where cache flush instructions are sometimes privileged, cache eviction is often too slow, and non-temporal stores are still cached in practice [79, 89, 11, 31] An important advantage of this method is that it does not require special privileges or permissions. Previous work takes advantage of Direct Memory Access (DMA) buffers exposed to userspace through the Android ION memory management interface [79]. GuardION [78], developed as an academic research project [81] by Van der Veen et al. [78] in 2018, aimed to mitigate Rowhammer attacks such as [79, 80] on ARM-based Android devices. Nonetheless, it was not integrated into the Android Open Source Project (AOSP) and therefore does not represent a deployed defense [81]. More generally, leveraging user-accessible DMA buffers on Android systems to bypass CPU caches has been explored in [79, 89]. Despite its effectiveness on Android, this technique suffers from platform-specific memory management limitations. The Android ION memory management framework is tightly coupled to the Android kernel and userspace stack and is not available on standard Raspberry Pi Linux distributions. Consequently, user-accessible DMA buffers provided by Android ION cannot be allocated or controlled on a Raspberry Pi, rendering this cache-bypass technique inapplicable in that environment.
In the context of this work, the previously discussed methods are therefore unsuitable or impractical. As a result, two methods remain of primary relevance for achieving uncached memory accesses on the target platform: cache eviction and cache flushing. These approaches are discussed in the following subsections.

5.3 Cache eviction

Cache eviction refers to the process of deliberately forcing cache lines out of the cache by accessing memory addresses that are mapped to the same cache eviction set. By repeatedly accessing such congruent addresses, an attacker can ensure that a specific target cache line is replaced and therefore removed from the cache.
In the context of Graphics Processing Unit (GPU) cache eviction, prior work has shown that by cleverly evicting the cache of an integrated GPU, access to main memory can be achieved

with significantly reduced latency, enabling memory accesses [67, 89]. This approach exploits the relatively simple cache structures of integrated GPUs, which can be evicted efficiently in order to achieve main-memory accesses [8, 50, 67].
CPU cache eviction is considerably more complex [59]. This complexity arises from higher cache associativity, the presence of multiple cache levels, and sophisticated cache replacement policies. Nevertheless, empirical studies demonstrate that CPU cache eviction is feasible in practice, provided that precise timing and detailed knowledge of cache set mappings are available [57, 39].
This section first outlines cache management, focusing on cache line replacement policies and cache inclusiveness. Finally, the section presents the development of cache eviction mechanisms, covering functionality, implementation details, and eviction strategies.

5.3.1 Cache Characteristics

table 2 and table 3 summarize the cache configurations of the Raspberry Pi 4 and Raspberry Pi 5, respectively. These tables provide an overview of cache sizes, associativity, cache levels, and the number of cache sets for the different cache hierarchies present on both platforms.

Table 2: RPi4 Cache using '`lscpu --cache`' command

NAME	ONE-SIZE	ALL-SIZE	WAYS	TYPE	LEVEL	SETS	COHERENCY-SIZE
L1d	32K	128K	2	Data	1	256	64
L1i	48K	192K	3	Instruction	1	256	64
L2	1M	1M	16	Unified	2	1024	64

Table 3: RPi5 Cache using '`lscpu --cache`' command

NAME	ONE-SIZE	ALL-SIZE	WAYS	TYPE	LEVEL	SETS	COHERENCY-SIZE
L1d	64K	256K	4	Data	1	256	64
L1i	64K	256K	4	Instruction	1	256	64
L2	512K	2M	8	Unified	2	1024	64
L3	2M	2M	16	Unified	3	2048	64

The number of cache sets is determined by the cache size, the cache line size, and the set associativity. This relationship can be expressed as

$$2^{\text{Index}} = \frac{\text{Cache Size}}{\text{Line Size} \times \text{Set associativity}}.$$

Applying this formula to the L2 cache of the Raspberry Pi 4 results in

$$\text{Number of sets} = \frac{1024 \times 1024}{64 \times 16} = 1024 = 2^{10}.$$

Similarly, for the L2 cache of the Raspberry Pi 5, the number of cache sets is given by

$$\text{Number of sets} = \frac{512 \times 1024}{64 \times 8} = 1024 = 2^{10}.$$

These calculations show that, despite differences in cache size and associativity, both systems employ the same number of cache sets at the L2 cache level.
On the Raspberry Pi 4, the L1 instruction cache, L1 data cache, and L2 cache behave as PIPT caches [43, 12]. On the Raspberry Pi 5, the L1 instruction and L1 data caches are VIPT, but they behave effectively as PIPT 4-way set-associative caches [16].

5.3.2 Cache Management

When a cache is full and a cache miss occurs, buffered code and data must be evicted from the cache to make room for new cache entries [59]. The heuristic used to decide which cache entry to evict is referred to as the cache replacement policy, which aims to select the entry that is least likely to be used in the near future [89]. The chosen replacement policy directly influences the number and access patterns required for an effective eviction strategy [57].
The Least Recently Used (LRU) replacement policy replaces the cache entry that has not been accessed for the longest time. In contrast, a pseudo-random replacement policy selects a cache entry to evict based on a pseudorandom number generator [57].
On the Raspberry Pi 4, the L1 instruction and L1 data caches use an LRU replacement policy, while the L2 cache employs a pseudo-LRU replacement policy [43, 12]. On the Raspberry Pi 5, the L1 instruction and L1 data caches use a pseudo-LRU replacement policy, and a dynamic biased replacement policy is employed [16].
In a multi-level cache hierarchy, it must be decided which cache levels store copies of data. A higher-level cache is called inclusive with respect to a lower-level cache if all cache lines present in the lower-level cache are also stored in the higher-level cache. Caches are considered exclusive if a cache line can reside in only one of two cache levels [58, 57]. If a cache is neither inclusive nor exclusive, it is referred to as non-inclusive.
On the Raspberry Pi 4, the L2 cache is inclusive with respect to the L1 cache [12]. On the Raspberry Pi 5, the cache hierarchy is strictly inclusive with respect to the L1 data cache and weakly inclusive with respect to the L1 instruction cache [16]. The L3 cache is neither strictly inclusive nor strictly exclusive, but rather non-inclusive, with a dynamic allocation policy that depends on the access pattern [13, 76]. Specifically, a first-time access by a core allocates data only to its L1 and L2 caches and not to the L3. If data is evicted from the L2 cache, it can be reloaded into the L3 cache, exhibiting exclusive behavior. If multiple cores share the same

cache line, it may reside simultaneously in the L3 cache and in multiple L2 caches, exhibiting inclusive behavior [13, 76].

5.3.3 Eviction Development

In this work we use the cache eviction mechanism based on the libflush source of the armageddon tool [33] proposed by Lipp et al. [59] and relies on the construction and use of eviction sets.

The eviction set structure, as defined in listing 1, consists of multiple virtual addresses that are congruent, meaning that they map to the same cache set. Accessing all addresses contained in an eviction set therefore ensures that a targeted cache line occupying the same set can be evicted from the cache.

```
typedef struct congruent_address_cache_entry_s {
  bool used;
  void* congruent_virtual_addresses[ADDRESS_COUNT];
} congruent_address_cache_entry_t;
```

Listing 1: Eviction set structure

The actual eviction procedure is implemented as a nested loop structure that repeatedly accesses the virtual addresses contained in an eviction set (see listing 2). During execution, all addresses are iterated over multiple times, thereby generating sustained pressure on the corresponding cache set. These repeated memory accesses are intended to reliably force the eviction of a target cache line. The behavior of the eviction routine is governed by several configuration parameters. Together, these parameters control both the repetition frequency of the eviction process and the coverage of memory accesses within the eviction set.

```
inline void evict(congruent_address_cache_entry_t*
   ↪ congruent_address_cache_entry) {
  for (unsigned int i = 0; i < ES_EVICTION_COUNTER; i += 1) {
    for (unsigned int j = 0; j < ES_NUMBER_OF_ACCESSES_IN_LOOP;
       ↪ j++) {
      for (unsigned int k = 0; k <
         ↪ ES_DIFFERENT_ADDRESSES_IN_LOOP; k++) {
        libflush_access_memory(congruent_address_cache_entry->
           ↪ congruent_virtual_addresses[i+k]);
      }
    }
  }
```

```
}
```

Listing 2: Function to evict cache lines for constructing eviction sets

To correctly construct eviction sets, it is necessary to determine the cache set index of a given memory address (see listing 3). This is achieved by computing the set index from the physical address of the memory location. The calculation is performed by shifting the physical address by `LINE_LENGTH_LOG2` bits, corresponding to the cache line size, and then taking the result modulo the total number of cache sets. This computation ensures that all addresses selected for an eviction set indeed target the same cache set, which is a prerequisite for effective cache eviction.

```
size_t libflush_eviction_get_set_index(libflush_session_t*
    ↪ session, void* address) {
  uintptr_t physical_address = libflush_get_physical_address(
      ↪ session, (uintptr_t) address);
  return (physical_address >> LINE_LENGTH_LOG2) % NUMBER_OF_SETS
      ↪ ;
}
```

Listing 3: Cache set determination

Libflush [33] was built for the ARMv8 architecture. A system-wide installation without Android dependencies was performed. The provided example application was subsequently built and executed which additionally generated measurement plots. To streamline the build process, the Makefile in `armageddon/libflush` was improved by introducing Raspberry Pi specific default build targets, allowing straightforward execution of `make` and `make install`. Furthermore, the Makefile in `cache-bypass` was modified for simplify the usage by forwarding these targets to `armageddon/libflush`. The `libflush/example` application was adapted to be directly executable and was initially used to evaluate experimental observations. It was then extended to include averaging over multiple repetitions to improve statistical robustness, automated sequential execution of all cache attack methods, persistent storage of measurement results in `.dat` files, and a Python-based solution for automated plotting and visualization. Finally, the adapted example was integrated into the local `cache-bypass` directory structure.

Cache attack techniques such as *Flush+Reload* and *Flush+Flush* typically rely on the unprivileged x86 `CLFLUSH` instruction to evict cache lines. In contrast, ARMv8-A CPUs do not provide an unprivileged flush instruction [79], which makes cache eviction strategies necessary instead [57]. In order to obtain the high-frequency measurements required by

modern cache attacks, eviction must be sufficiently fast, as previously proposed eviction strategies have been shown to be too slow [69]. The Lipp et al. [59, 33] approach is applicable only to devices with a known eviction strategy [50], and therefore the default configuration provided by the libflush tool [33] is adopted (see listing 4).

```
#define NUMBER_OF_SETS 4096
#define LINE_LENGTH_LOG2 6
#define LINE_LENGTH 64
#define ES_EVICTION_COUNTER 28
#define ES_NUMBER_OF_ACCESSES_IN_LOOP 5
#define ES_DIFFERENT_ADDRESSES_IN_LOOP 4
```

Listing 4: Parameter definitions for eviction set configuration

The relevant configuration parameters are defined in the default libflush [33] configuration. The parameter `NUMBER_OF_SETS` specifies the total number of cache sets considered for eviction and determines how addresses are indexed to cache sets. The parameters `LINE_LENGTH_LOG2` and `LINE_LENGTH` define the cache line size of 64 bytes and its base-2 logarithm, which are required for address calculations. The parameter `ES_EVICTION_COUNTER` determines the number of addresses in an eviction set that are accessed per iteration in order to reliably evict a target cache line.
The parameter `ES_NUMBER_OF_ACCESSES_IN_LOOP` specifies how often the eviction loop is repeated to ensure eviction under noisy conditions.
Finally, `ES_DIFFERENT_ADDRESSES_IN_LOOP` defines the number of distinct addresses accessed in each inner iteration of the eviction loop, thereby distributing accesses across the cache set to achieve robust eviction behavior.

5.4 Flush cache

We recommend this method for bypassing the cache because it explicitly invalidates a specific cache line and thereby forces the next access to be served from DRAM. ARMv8 provides explicit cache maintenance instructions for this purpose, most notably the `DC CIVAC` instruction [50]. By directly targeting individual cache lines, this approach provides deterministic cache-line invalidation, which guarantees predictable behavior whereby every subsequent access is fetched from DRAM. It should be noted that these instructions are no longer available in an unprivileged context but require privileged execution. Even so, this restriction is acceptable for our project. Overall, this technique avoids reliance on undocumented cache-replacement policies, thereby increasing experimental reliability, and simplifies the setup by

eliminating the need for eviction-set construction and timing-based profiling that are typical of cache eviction based approaches.

5.4.1 Instruction Types

The original Rowhammer approach targeting the x86 architecture relies on the unprivileged `CLFLUSH` instruction to flush the data associated with a given address from the cache hierarchy [87, 89]. While cache management instructions in ARMv7-A, referred to as cache maintenance instructions, are privileged in all ARMv7-A processors, they have been exposed to userspace since ARMv8-A. Consequently, it becomes feasible to apply Rowhammer techniques similar to the original CLFLUSH-based approach on systems using ARMv8-A processors [89], as descibed in fig. 4.

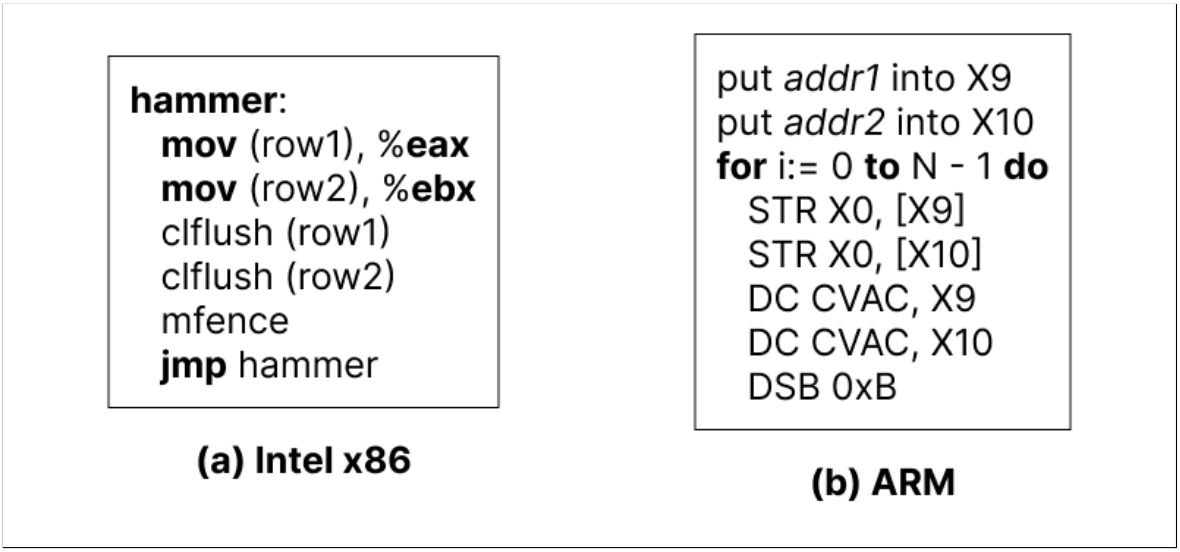

Figure 4: Flush instructions on x86 and ARM [89]

In the Intel x86 architecture, the `CLFLUSH` instruction invalidates the cache line containing the specified address from all levels of the cache hierarchy, including both data and instruction caches [57]. If the cache line is dirty at any level of the hierarchy, meaning it contains modified data, it is written back to memory before being invalidated [28, 32, 57]. Both `CLFLUSH` and its related instructions are unprivileged and therefore accessible from userspace [32, 57].
ARMv8-A defines several cache maintenance instructions with distinct semantics. The `DC CIVAC` instruction cleans and invalidates a data cache by address to Point of Coherency (PoC) [14], while `DC CVAC` cleans a data cache line by address to the PoC without invalidation [15]. In addition, ARMv8-A introduces the `DC ZVA` instruction, which is designed to efficiently zero out a block of memory [89]. This instruction zeros a naturally aligned block of N bytes [19]. As described in [86], the capability to initialize the data cache with zeroed

values through the DC ZVA instruction was introduced in the ARMv8-A architecture. Cache line zeroing operates analogously to a prefetch mechanism by signaling to the processor that specific memory addresses are expected to be accessed subsequently. Nevertheless, the zeroing process can be significantly faster, as it eliminates the requirement to await the completion of external memory accesses [89].

When using DC ZVA in the context of cache bypassing or Rowhammer-style experiments, a crucial requirement is that the memory blocks being zeroed must not already be cached in the hierarchy before entering the hammering loop. Otherwise, the underlying DRAM will not be accessed; instead, only the cached copies are zeroed and marked as dirty [89]. This behavior is fundamentally different from the use of non-temporal store instructions on x86 and would invalidate the intended experimental assumptions [89]. The corresponding access pattern and instruction sequence [89] are illustrated in Figure 5.

```
put addr1 into X9
put addr2 into X10
invalidate/evict addr1
invalidate/evict addr2
for i := 0 to N - 1 do
  DC ZVA, X9
  DC ZVA, X1
```

Listing 5: Cache line zeroing using DC ZVA

5.4.2 Kernel Module

We investigate the cache bypass mechanism further using a Linux kernel module designed for kernel-space latency measurements of memory and cache operations on ARMv8 systems. The primary goal of this module is to determine whether cache hits and cache misses can be statistically distinguished and to visualize this distinction by plotting latency distributions for cache hits versus cache misses.

The module uses the CPU cycle counter pmccntr_el0 in combination with explicit instruction and data synchronization barriers to measure the latency of individual operations in CPU cycles. It implements eight different operation modes, each representing a specific memory access pattern, including store and load operations, cache clean operations using DC CVAC, cache clean and invalidate operations using DC CIVAC, and cache line zeroing using DC ZVA. We further evaluate each of these modes with and without explicit synchronization, thereby intentionally exposing different ordering and completion behaviors. In the context of CLFLUSH based Rowhammer attacks on x86 architectures, Xiao et al. [88] observed that the exclusion

of memory barrier instructions, such as Data Synchronization Barrier (DSB) and Instruction Synchronization Barrier (ISB), leads to a higher effectiveness in triggering bitflips [89]. Consistent with these findings, [89] reports similar observations when conducting comparable experiments on ARM-based systems. Therefore not all evaluated modes enforce full memory barrier instructions after memory operations. This design choice in our experiment enables an analysis of the latency effects resulting from the absence versus the enforcement of such barriers. A baseline mode measures plain memory loads to serve as a reference for comparison against all cache maintenance modes. We execute each configuration repeatedly and average the results over multiple runs to reduce noise. The resulting latency distributions are written to a file and intended to be plotted as diagrams, enabling a clear comparison between modes and highlighting the influence of synchronization on latency and measurement comparability. A notable issue encountered during these experiments is the occurrence of zero latency readings when using cache-bypass kernel modules, despite an active PMU and relaxed paranoid settings. This behavior is observed specifically on Raspberry Pi 5 systems, and its underlying cause remains unresolved at this stage, as the issue affects only the kernel module on all Raspberry Pi 5 devices, despite it having previously worked correctly. Userspace functionality remains unaffected.

In contrast to the ARMv7 architecture, ARMv8 defines the `SCTLR_EL1` system control register, which includes a bit denoted as `UCI`. If this bit is set, userspace access is enabled for a specific set of cache and instruction maintenance operations, as illustrated in table 4 [57, 11].

Table 4: Instructions affected by the UCI bit in SCTLR_EL1

Instruction	**Description**
DC CVAU	Data or Unified Cache line Clean by VA to PoU
DC CIVAC	Data or Unified Cache line Clean and Invalidate by VA to PoC
DC CVAC	Data or Unified Cache line Clean by VA to PoC
IC IVAU	Instruction Cache line Invalidate by VA to PoU

The `SCTLR_EL1` register is a 64-bit system control register that provides top-level control of the system, including its memory system, at both EL1 and EL0 [17, 12]. The `UCI` field is located at bit position `26` [17, 18] and enables EL0 access to the aforementioned cache maintenance instructions when set to 1, while a value of 0 disables EL0 access [18]. In addition, `SCTLR_EL1` contains the `Data Zeroing Extension (DZE)` field at bit position `14` [17, 18]. This bit controls EL0 access to the `DC ZVA` instruction, where a value of 1 enables execution in userspace and a value of 0 disables it [18].

To verify the configuration of these fields, a dedicated kernel module `sctlr.c` was implemented to read and report the contents of `SCTLR_EL1`. The module output confirms that

both the UCI and DZE bits are set. This result demonstrates that DC CIVAC and DC CVAC are unprivileged and can be executed from userspace, and that DC ZVA is likewise unprivileged and accessible at EL0. These findings are consistent with previous observations reported in [89].

5.4.3 Userspace

A userspace counterpart to the kernel module experiments is implemented in the program flush_user.c. The goal of this experiment is likewise to determine whether cache hits and cache misses can be statistically distinguished and to visualize the difference by plotting corresponding latency distributions.

This experiment evaluates the effectiveness and performance impact of cache maintenance instructions on ARM-based systems when executed from userspace. The objective is to assess whether, and to what extent, specific instructions can bypass or influence cache behavior, and to quantify the associated latency costs. The program measures cycle-accurate execution times using ARM performance counters while applying different low-level memory operations. It executes eight predefined instruction patterns involving store, load, clean, invalidate, and zeroing operations. We executed each pattern repeatedly under two conditions: with and without the corresponding cache maintenance instruction.

Compared to the kernel-space implementation, several differences arise. The kernel-based flush_kernel.c executes in kernel space as a loadable module using module_init and module_exit, providing direct access to hardware resources and privileged instructions. In contrast, flush_user.c runs entirely in user space as a standard program with a main() function and without kernel privileges. Memory management also differs, as the kernel module relies on kmalloc() and kfree() to obtain physically contiguous memory, whereas the userspace program uses standard virtual memory allocation mechanisms such as malloc() and free(). File I/O and logging are handled using kernel-specific interfaces in kernel space, while the userspace implementation uses standard C library functions.

6 Address Mapping Reconstruction

After bypassing the cache hierarchy, a fundamental challenge remains, namely the existence of a fixed mapping between physical addresses and the DRAM layout, as noted in [39]. To achieve "*C2. Address Mapping Reconstruction*", it is necessary to access a specific DRAM row within the same bank in order to gain bitflips in an adjacent victim row through produced row conflicts. Notwithstanding, the exact informations of this physical-to-DRAM address mapping are typically not publicly available. While AMD provided documentation of the DRAM bank addressing functions for older processor models - specifically in the BIOS and Kernel Developer's Guide (BKDG) up to microarchitecture 16h [3] - they ceased to publish this information with the beginning of microarchitecture 17h (released in 2017 [2]) [45]. Unlike AMD's earlier documentation efforts, Intel has never disclosed the DRAM addressing functions implemented in its processors [45],which is also the case for all our ARM test devices used in this work. Consequently, the only viable approach is to reverse engineer the DRAM address mapping. This section briefly discusses the available methods, explains why certain approaches are adopted or discarded, and identifies the most effective technique for ARM-based systems such as the Raspberry Pi. In addition, it outlines the practical implementation process, including the development steps and the challenges encountered during implementation.

6.1 Methods for Inferring DRAM Organization

If the internal organization of DRAM and the behavior of the row buffer are known, it becomes feasible to identify two distinct rows located within the same bank. Based on this observation, several approaches have been proposed to reverse engineer the mapping between physical addresses and DRAM structures. These approaches mainly differ in how address sets are constructed and how the underlying addressing functions are inferred. This work focuses on three representative methods, which are listed below, along with a brief description of each.

Pessl et al. [66] in 2016, exploits a row-buffer timing side channel in which row conflicts manifest as increased memory access latency. By observing row hits and row conflicts, physical addresses can be grouped into sets that reside within the same bank and thus share identical channel, DIMM, rank, and bank properties. The underlying DRAM address mapping is inferred by exhaustively evaluating linear addressing functions that distinguish these sets, which are commonly expressed as XOR combinations of physical address bits. This general methodology underlies several subsequent attacks and tools, including [6, 36, 79, 80, 57].

The work introduced by Xiao et al. [88] in 2016, proposes a graph-based approach to infer physical-to-DRAM address mappings. Similar to DRAMA, it relies on timing differences caused by row conflicts, but its primarily aimed at enableing Rowhammer attacks across virtual machines.
Plin et al. [9] in 2025, reformulates the identification of DRAM parity masks as a linear algebra problem as Helm et al. [21] in 2020. Instead of relying on exhaustive search, this method focuses on solving the address mapping directly through algebraic reasoning.
The following sections assess the suitability of these methods by applying them to the test systems and confirming or refuting them based on experimental results.

6.1.1 Graph-Based Mapping Inference

Building on the timing-channel primitive introduced in [88], we employ a novel graph-based algorithm to determine the memory mapping at runtime. This approach models each bit in a physical address as a node in a graph and establishes relationships between nodes based on measured memory access latencies. By exploiting the fact that specific latency patterns emerge when addresses differ in particular bit positions, the method enables the automatic and accurate detection of row bits, column bits, and bank bits.
The core idea [1] is implemented using a kernel module, named `latency.c` which reverse-engineers the physical-to-DRAM address mapping by measuring access latencies `L(i,j)` between pairs of physical addresses that differ in bits `i` and `j`. Large access latencies occur only when the two addresses reside in different rows within the same bank, which allows the identification of row, column, and bank bits. On ARMv8 systems, it specifically focuses on locating the second-lowest row bit by systematically measuring the latency between addresses that differ in exactly two bits and inferring which bit positions correspond to row differences [1].
The program output reports the time required to access two physical addresses that differ in two bit positions. Each measurement is represented in the form `L(i,j):T`, where `T` denotes the measured access time when the addresses differ in bit `i` and bit `j`. In the special case `i=j`, the addresses differ in only a single bit. The key observation exploited by the analysis is that high latency values occur exclusively when two addresses are located in different rows of the same bank, which provides a distinguishing feature for identifying row, column, bank bits.
The workflow involves porting, building and executing the tool [1] by including several ARM-specific adaptations for performance counters and cache flush instructions. In addition, dynamic memory allocation via `vmalloc` was introduced to accommodate larger arrays. The corresponding Python script stores the results in `latency.log` file and analyzes all bit

positions. The three bits exhibiting the highest average latency deviations are identified. Analysis across all bit positions revealed that bits 12, 13, 14 consistently exhibit the highest average latencies, while bits 15 and 16 appear occasionally, as seen in fig. 5.

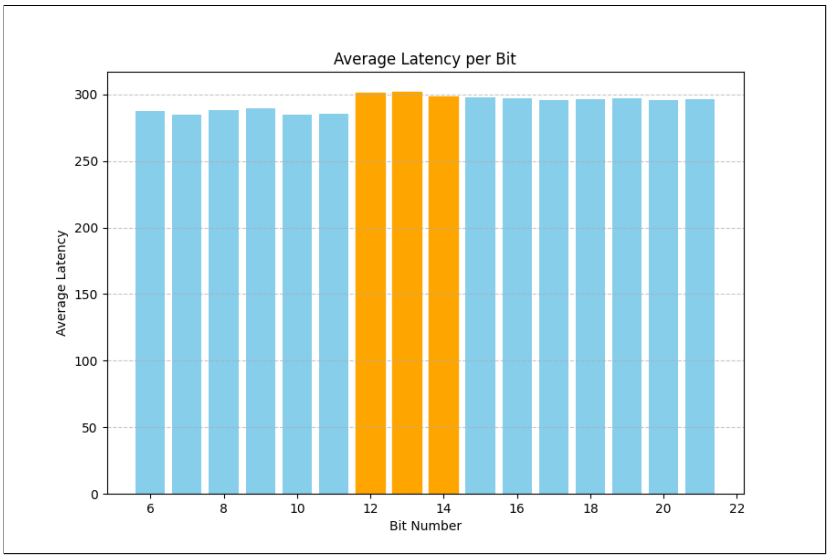

Figure 5: Average latency per bit position

Despite these results, a problem arises from the fact that the observed latency deviations are relatively small and do not demonstrate a significant difference. Despite this limited magnitude of deviation, the resulting mapping remains consistent across multiple measurements. This consistency aligns with findings from related work, particularly the mapping of [5], as well of the [49] mapping. However, due to the lack of pronounced latency differences, these observations alone were insufficient, and consequently additional methods were explored.

6.1.2 Algebraic Mapping Inference

An alternative approach to physical-to-DRAM mapping inference is based on algebraic techniques described in [9]. As outlined in the previous section of the targeted approach, [9] treats identifying parity masks as a linear algebra problem. This technique directly targets the physical-to-DRAM mapping and enables the recovery of bank and row functions without relying on iterative function checks. Instead, it computes the nullspace of conflicting address sets.

A major advantage of this tool [10] is that it is a platform-agnostic design. It supports a wide range of architectures, including x86_64, ARMv8, and POWER9/10, which makes it suitable for all test devices used in this work.

The workflow requires activation of the PMU via a kernel module. No matter how the workflows were executed, both the automatic and manual approaches failed to recover any

valid mapping. In all cases, the execution terminated with the warning "*Threshold quality check failed: Valley between peaks not pronounced enough.*" Multiple mitigation attempts were undertaken, including manual threshold selection, parameter tuning for noisy systems, and high-precision measurement runs. Nevertheless, all attempts resulted in an insufficient number of coherent conflict samples, which caused the derivation of bank masks to abort and the row mask analysis to be skipped entirely, as seen in fig. 6.

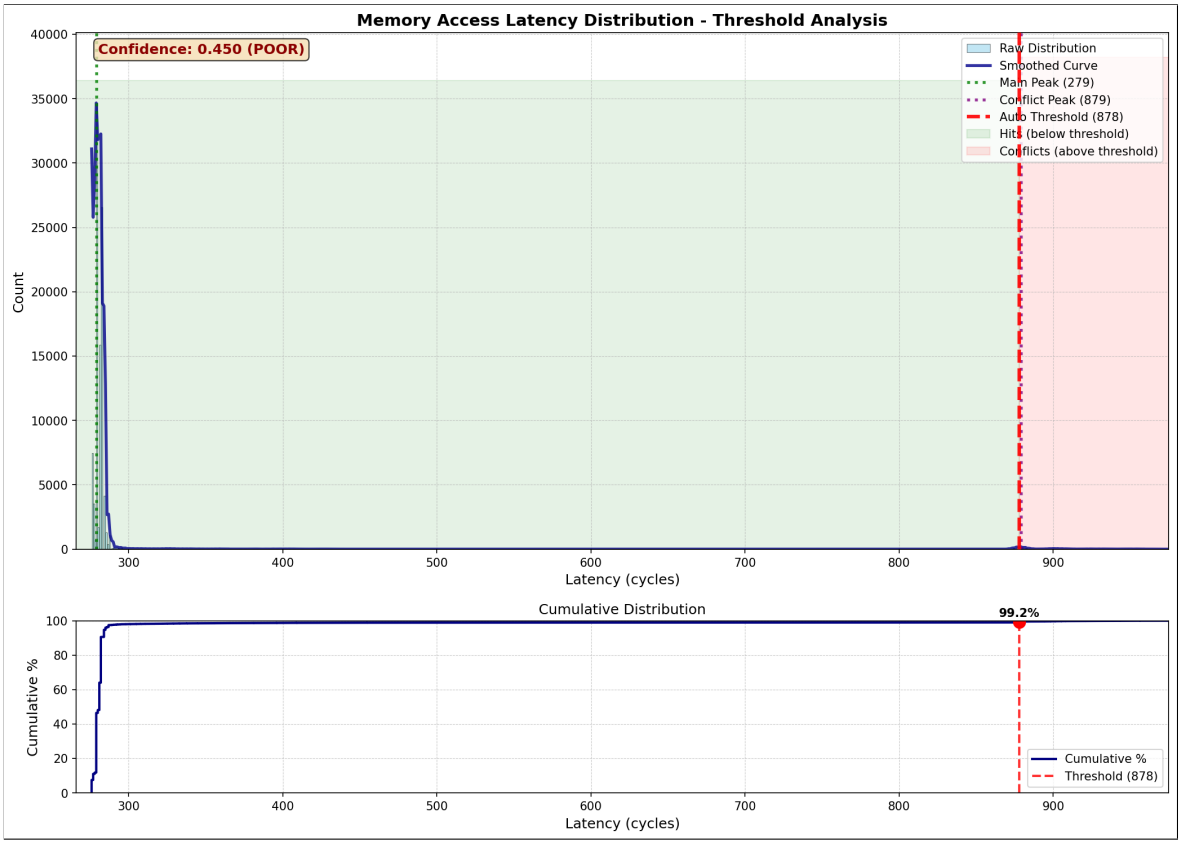

Figure 6: Memory access latency distribution with threshold analysis

This limitation is closely related to the employed closed-page policy [47]. Even the authors of [9] reported that they were unable to retrieve any part of the mapping on a Raspberry Pi 4, as the observed latency distribution consisted of a single peak indicative of a closed-page policy. The same behavior was observed not only on the tested Raspberry Pi 4 but also consistently across all Raspberry Pi 5 devices examined in this work. Further details and implications of this issue are discussed in "Discussion and Limitations" of [37].

Due to this limitation, we briefly adopted a closed page policy in theory. Since many tasks require an open page policy, we had to find another way to trigger the Rowhammer bug. Based on the findings of Gruss et al. [25], who showed that the memory controller's page policy significantly influences how the Rowhammer vulnerability can be triggered, we were able to pursue an approach aimed at exploiting the Rowhammer bug even under a closed-page

policy, which was further confirmed by [60]. This approach is based on the realization that one-location hammering is also possible with a closed side policy [60, 25]. To implement this approach, we used the FlipFloyd tool foundation [35] and adapted it to our ARM-based systems by changing the cache flush instructions and integrating the performance counter `PMCCNTR_EL0`. This enables us to carry out the attack without having to know the exact physical DRAM address mapping.

6.1.3 Row Hit, Row Conflict Distinction

Distinguishing row hits from row conflicts is a fundamental requirement for reverse engineering DRAM address mappings, because access latency directly reflects the internal organization of banks and rows. By correlating timing differences with chosen address pairs, one can infer which physical address bits select the bank and which select the row, without vendor documentation. Based on the issues encountered in the previous part - most notably the insufficiently pronounced valley between latency peaks - it remains unclear whether a reliable distinction between row hits and row conflicts is feasible in practice.
To further investigate this problem, an approach was designed to explicitly analyze the distinction between row hits and row conflicts. This approach builds directly on observations from DRAMA and similar prior work. Since access latency reflects the internal structure of banks and rows, measuring timing differences between address pairs can reveal which physical address bits are responsible for bank selection and which correspond to row selection, without relying on proprietary information. Three types of access patterns are considered [66].
Bank Miss (see fig. 7a): both addresses are mapped to different banks. Since accesses to separate banks do not interfere, this results in low latency, analogous to a Row Hit but across banks.
Row Hit (see fig. 7b): both addresses reside in the same bank and row, also leading to low access times.
Row Conflict (see fig. 7c): both addresses are in the same bank but different rows, which results in high access times.

Assuming a system with N banks and R rows per bank, the probability P of a bank miss is given by:

$$P_{\text{Bank Miss}} = 1 - \frac{1}{N}.$$

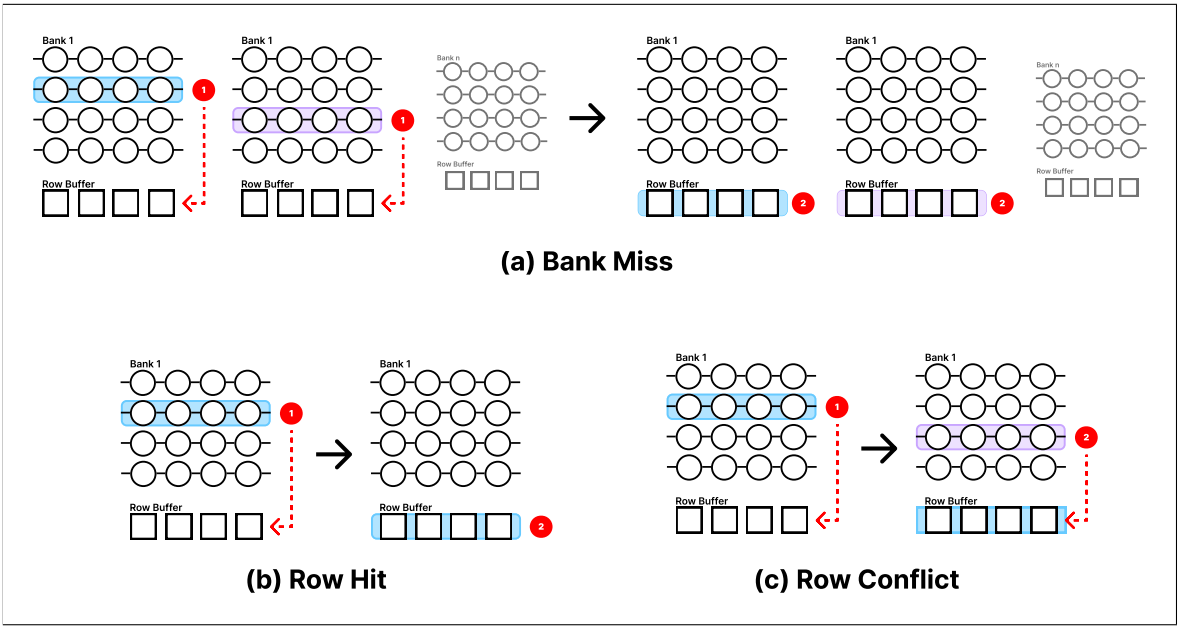

Figure 7: Types of access patterns

The probability of a row hit is:

$$P_{\text{Row Hit}} = \frac{1}{N} \cdot \frac{1}{R}.$$

while the probability of a row conflict is:

$$P_{\text{Row Conflict}} = \frac{1}{N} \cdot \left(1 - \frac{1}{R}\right).$$

Under these assumptions, two peaks are expected to become visible in the latency distribution. The left peak corresponds to low access times and consists of bank misses and row hits, while the right peak corresponds to high access times and consists of row conflicts [26, 66]. To accurately distinguish between row hits and row conflicts in CPU, it is essential to measure memory access latencies while eliminating the influence of CPU caches. We achieve this by isolating CPU from cache effects, which includes pinning the CPU to a single core and repeatedly accessing pairs of randomly selected addresses, identified through pagemap. Between each access, we are executing cache flush instructions such as `DC CIVAC`, and probing each address pair for approximately 100.000 iterations. The access latencies between these addresses are recorded, and after averaging across multiple measurements, the results are consolidated into a single `.log` file. From this consolidated data, histograms representing the distribution of row hits and row conflicts are generated, which helps reduce measurement noise.

We implemented the measurement at different temporal resolutions. In the low-resolution approach, as demonstrated in `bank-timing-probe.c`, we used a POSIX function to record the elapsed time between events with nanosecond precision. However, the limited temporal resolution of `clock_gettime()` makes it unreliable for distinguishing the short latency differences between row hits and row conflicts (see fig. 8). To address this limitation, a high-

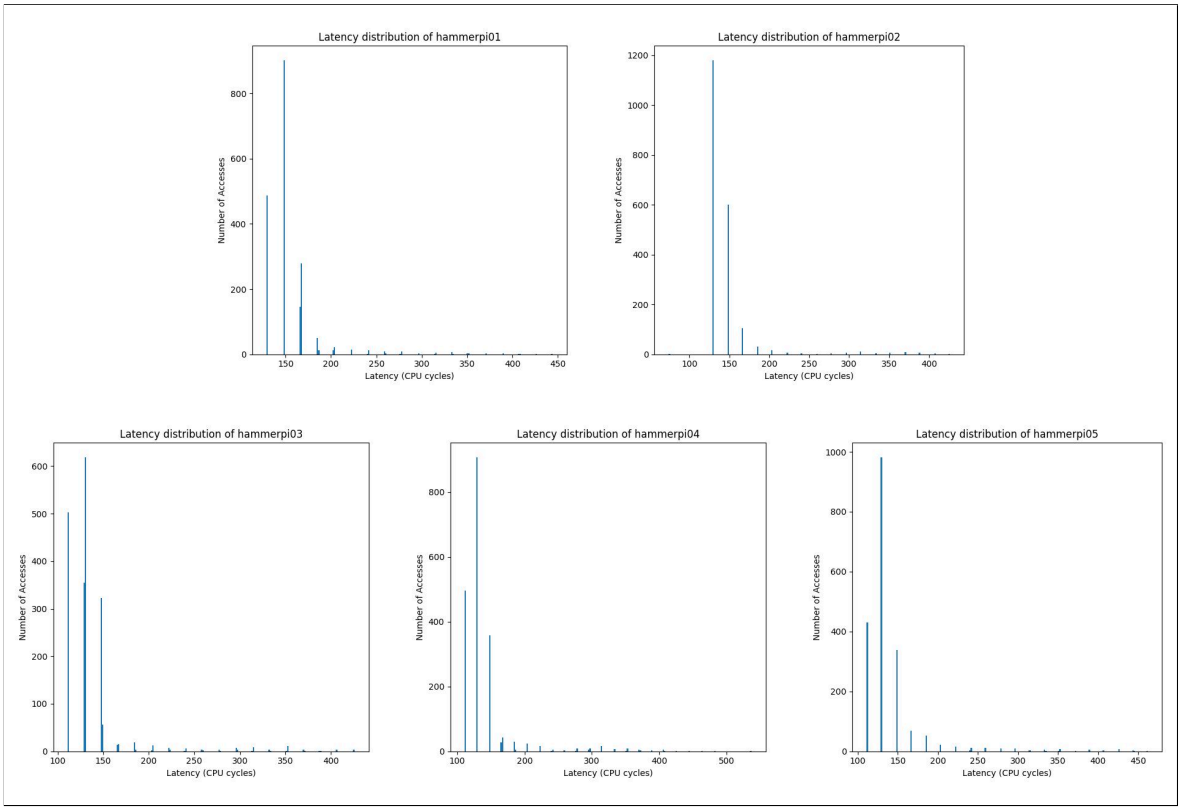

Figure 8: Low resolution with POSIX function `clock_gettime()`

resolution method, exemplified in `rowhitconflict.c`, employs the Performance Monitoring Cycle Count Register (PMCCNTR) instead of `clock_gettime()`, allowing for fine-grained measurement resolution.

Once we collected latency data, statistical analysis is performed to distinguish row hits from row conflicts reliably. This analysis involves evaluating the latency distributions, determining thresholds, and performing statistical validation, including computation of means, variances, confidence intervals, and t-tests. The methodology leverages prior insights from [66, 9].

A key step is identifying a reliable latency threshold that separates row hits from row conflicts. This is achieved by collecting thousands of latency samples and constructing a smoothed histogram. The main peak corresponding to row hits and the high-latency peak corresponding to row conflicts are identified, and the threshold is set at the "left foot" of the high-latency peak. We further validated the threshold by analyzing the peak distance and valley depth. If

validation fails, a fallback threshold is applied. This threshold is then used to classify memory accesses as row hits or row conflicts.
For statistical validation, we divided the latency samples into row hit and row conflict groups according to the threshold index. The results including mean, variance, confidence intervals, and the statistical significance of the separation, are printed to ensure that the distinction between row hits and row conflicts is statistically robust.

6.1.4 Bank Function Derivation

The DRAMA approach [66, 34, 50] derives DRAM address mappings by grouping physical addresses into DRAM sets based on access-time differences caused by row hits, row conflicts, and bank conflicts. From these sets, the tool infers the DRAM address mapping by identifying XOR combinations of physical address bits that consistently characterize the sets.
DRAMA has previously been demonstrated to work on several ARM-based platforms, including ARMv7 systems with LPDDR2 and LPDDR3 memory [66] and ARMv8-A systems with LPDDR3 [40] and LPDDR4 [41] memory [66, 50]. These prior results provide proof that DRAMA can be ported to ARM architectures. Several challenges were encountered during the ARM port.
First, the original DRAMA implementation relied on x86-specific instructions such as `RDTSC` and `CLFLUSH`, which are unavailable on ARM. To address this, these functions were implemented using `PMCCNTR_EL0` for precise timing, and a flush cache line function with a selection for one of eight ARM cache maintenance options, similar to [1], was added to replace `CLFLUSH`. The `-march=armv8-a` compiler flag was used to ensure optimized ARM code generation.
A second challenge concerned measurement stability. The original DRAMA code measured each memory access only once, which made the results susceptible to noise. To improve robustness, repeated measurements were introduced by defining multiple runs of the entire measurement process. For each run, the system initializes the memory mapping and random address pools, measures access times across all candidate addresses, and accumulates the observed XOR combinations. After all runs, the observed combinations are aggregated, sorted, and saved to `combinations.dat`, thereby increasing statistical confidence and reducing the influence of outliers.
In fact, two DRAMA tools were ported, which, while similar in purpose, differ in execution. The original DRAMA tool does not utilize hugepages, whereas the DRAMA implementation within TRRespass relies on hugepages for memory allocation.
Additionally, result visualization was improved. The original DRAMA code only printed XOR

functions and masks to the terminal, without any automated post-processing or visualization. To address this, all detected XOR and non-XOR combinations are counted in `measure.cpp` and written to `combinations.dat`. A Python script, `plot.py`, reads this file, separates XOR and non-XOR functions, sorts bit combinations numerically, plots the counts as a bar chart with distinct visual encodings, and saves the resulting figure with a filename derived from the host system. The Makefile integrates this workflow by executing the measurement with CPU pinning and automatically invoking `plot.py`, enabling reproducible and device-independent visual analysis of DRAM bank bit combinations.

As a final step, a bit mask check is used to verify a given physical-address-to-bank mapping using configurable bit masks. Bank address bits are defined either as single physical address bits or as XOR combinations of bits. A configurable fraction of system memory is allocated using `mmap`, and virtual-to-physical address translation is resolved via pagemap. For each physical address, the bank index is computed according to the specified bit mask mapping, and addresses belonging to the same bank are grouped into lists. The number of physical addresses per bank list is then output as a consistency check.

The results of this method strictly reflect the assumed bit-mask mapping provided as input. Consequently, the approach allows only a plausibility and internal-consistency check of bank assignments and does not guarantee that the derived mapping corresponds to the actual hardware bank mapping.

6.2 Verification of Reverse-Engineered DRAM Mapping

A DRAM mapping has been obtained for each test device. Nevertheless, it remains uncertain whether the reverse-engineered mapping is entirely correct (as seen in the previous part). Several methods can be employed to verify the correctness of the derived DRAM addressing functions.

One approach is **physical probing**, which involves comparing the determined functions with results obtained through direct hardware measurements, as described in [66]. Nonetheless, this method faces significant practical limitations. Embedded systems often incorporate small chips and multi-layered circuit boards [50], making direct probing difficult. In particular, verification via hardware probing is not feasible on devices such as Raspberry Pi's due to the presence of soldered DRAM modules.

Another verification method leverages **Rowhammer** testing. In this approach, correctness is assessed by evaluating whether the use of the addressing functions in Rowhammer experiments increases the number of bitflips per second proportionally to the number of sets identified [66]. This method is often impractical because tests may require excessively long

durations, the systems under study may not be vulnerable, or mitigations may interfere with the results.

The final method is a **software-based** verification, which entails performing timing measurements across a larger set of addresses to detect differences consistent with the derived addressing functions [66, 46]. This approach is feasible and provides a practical means of validating the correctness of the reverse-engineered DRAM mappings.

6.2.1 Software-based Approach

We conducted the verification of DRAM mapping by using a software-based, latency-driven methodology, following the approach proposed in the DRAMA methodology [34]. This method's considerations are based on the cases that the system can encounter - specifically bank misses, row hits, and row conflicts, as defined in the reverse engineering process. Each of these scenarios exhibits distinct access latencies, which can be exploited to verify the underlying DRAM mapping. The verification procedure involves several steps. First, it identifies memory addresses that reside within the same or different DRAM rows and banks, as shown in listing 6. Next, we measure access latencies after flushing the relevant cache lines to ensure that the measurements reflect DRAM behavior rather than cached values. Then we organize the collected latency data into histograms, which allow for the classification of row hits and row conflicts. These timing histograms serve to calibrate a latency threshold, enabling the verification of the DRAM mapping. Specifically, addresses predicted to reside within the same row should exhibit low access latencies, while addresses predicted to be located in different rows are expected to show higher latencies. This workflow faces challenges similar to those encountered in the previous DRAMA-based reverse engineering process. Key difficulties include ensuring portability across ARM architectures and mitigating measurement noise to maintain reliable latency distinctions.

```
1  p = base_address
2
3  // Find second address within same bank and row
4  for each address in memory_array:
5      if dram_bank(address) == dram_bank(p) and dram_row(address)
       ↪ == dram_row(p):
6          p2 = address
7          break
8
9  // find third address whitin same bank but different row
10 for each address in memory_array:
```

```
    if dram_bank(address) == dram_bank(p) and dram_row(address)
       ↪ != dram_row(p):
        p3 = address
        break
```

Listing 6: Finding congruent DRAM addresses for eviction sets

6.2.2 Rowhammer-based Approach

A secondary approach was tested to verify the DRAM mapping using the Rowhammer technique. This method aims to validate the addressing functions by observing whether their application leads to an increased rate of bitflips in memory.

We performed memory allocation `mmap`, based on a predefined percentage of available system memory, specified by `MEMORY_IN_PERCENT`. Then the MMU translated the virtual to physical addresses using pagemap. We grouped all of our physical addresses that correspond to the same DRAM bank into lists according to our reverse-engineered DRAM mapping. The hammering procedure iterates until a predefined number of cycles is reached, as denoted as `TIMES`. In each iteration, two physical addresses from the same bank are selected randomly. These addresses are repeatedly accessed hammered for a defined number of times while explicitly flushing the cache between accesses. After each iteration, the system checks for resulting bitflips. Once the procedure completes, the total number of detected bitflips is verified and reported.

The grouping of physical addresses follows the goal of assigning physical addresses to DRAM banks according to the reverse-engineered mapping. These addresses are partitioned into bank-specific lists, where each list contains addresses mapping to the same DRAM bank. These lists form the candidate sets used for the hammering process. The bank index is computed using the algorithm shown in listing 7.

```
int compute_bank(uint64_t pa) {
    int bank = 0;
    for (size_t b = 0; b < NUM_BANKS; b++) {
        int bit = 0;
        for (size_t i = 0; i < hLens[b]; i++) {
            bit ^= (pa >> hArrays[b][i]) & 1;
        }
        bank |= (bit << b);
    }
    return bank;
```

```
}
```

Listing 7: Algorithm for grouping physical addresses that yield the same bank index

During the hammering phase, two addresses from the same bank are repeatedly accessed while the cache is explicitly flushed after each access. This maximizes DRAM row activations and is intended to induce bitflips. The core idea is to randomly select two distinct addresses within one bank and repeatedly toogle between both in sequence, flushing the cache lines after each access, as illustrated in listing 8. We repeated this process for a large number of cycles.

```
for (size_t j = 0; j < HAMMER_CYCLES; ++j) {
    asm volatile(
        "str %2, [%0]\n\t"
        "str %2, [%1]\n\t"
        "dc cvac, %0\n\t"
        "dc cvac, %1\n\t"
        ::"r" (addr1), "r" (addr2), "r" (init_val)
    );
}
```

Listing 8: Hammering Mechanism

Device	Mappings	Bank Functions
Raspberry Pi 4B	Memory Aware DOS [49]	(11), (12), (13), (14)
Raspberry Pi 4B	DRAM MaUT [5]	(12), (13), (14), (11 XOR 12)
Raspberry Pi 3B+	Flipping Bits Like a Pro [4]	(13 XOR 14), (14), (15)
Exynos 7420	DRAMA [66]	(14), (15), (16)

Table 5: Related Work Mappings

Multiple attacks were conducted to evaluate both the mappings proposed in related work, as seen in table 5, and the reverse-engineered mappings, with each Raspberry Pi device tested for approximately five hours. Across all tests, no bitflips were observed. The resulting mapping is likely correct, as it closely resembles mappings reported in other studies for the Raspberry Pi 4 and older models. Repeated measurements consistently produced the same mapping. The absence of bitflips indicates that either hardware or software mitigations effectively prevent Rowhammer-induced errors, or the DRAM modules under test are inherently resistant. Therefore, based on the obtained results, no definitive conclusions regarding Rowhammer susceptibility can be drawn for these devices.

7 Outpacing DRAM Refresh

To achieve "*C3. Outpacing DRAM Refresh*", memory accesses must race against the next row refresh in order to induce sufficiently frequent activation's of the same DRAM rows. Accordingly, this section focuses on the implementation of access patterns and parameter configurations for high-frequency activation with some optimization techniques, and adopts concrete techniques for handling and bypassing mitigation mechanisms to sustain effective hammering on ARM-based systems such as Raspberry Pi devices.

7.1 TRR Detection Approaches

TRR is a standard hardware protection mechanism designed to successfully prevent both single-sided and double-sided Rowhammer attacks [85, 68]. When a Rowhammer attack targets a specific row, TRR responds by generating an additional refresh operation to refresh the corresponding victim row, thereby mitigating disturbance effects [24]. The primary objective of this analysis is to determine whether TRR is available and active on the test systems used in this work.

To investigate this, the official JEDEC documentation for LPDDR4 [41] and LPDDR4x [42] was examined first. This analysis revealed that the section describing TRR mode had been removed from the current official documentation. Nevertheless, it was noted that the Mode Register 0 configuration specifies that TRR support is indicated when `MR0 OP[2]` is set to `0b`. In parallel, the Raspberry Pi devices were physically inspected by unscrewing them in order to identify the DRAM manufacturer, the component markings, and the FBGA numbers. Using the "FBGA to component marking decoder" provided on the official manufacturer website [52], the exact part numbers of the DRAM components were determined, as summarized in table 6.

Table 6: FBGA Part Number Decoder

Model	RAM	Manufacturer	FBGA	Part Number
Raspberry Pi 4	4 GB	Micron	D9WHV	MT53D1024M32D4DT-053 WT:D
Raspberry Pi 4	8 GB	Micron	D9ZCL	MT53E2G32D4NQ-046 WT:A
Raspberry Pi 5	4 GB	Micron	D8CJG	MT53E1G32D2FW-046 WT:B
Raspberry Pi 5	8 GB	Micron	D8CJN	MT53E2G32D4DE-046 WT:C
Raspberry Pi 5	16 GB	Micron	D8CBG	MT53E4G32D8CY-046 WT:C

The identification of the exact DRAM components enabled conclusions to be drawn regarding the officially applicable documentation for the test devices [55, 53, 54, 56]. However, this documentation could not be accessed, even after contacting the manufacturers directly, as

no response was received. Despite this limitation, an older Micron reference document from 2022 was identified [51], which provides sufficient information about the specific devices used in this work. Importantly, this reference still includes the TRR Mode section that has been removed from newer documentation.

From this reference, the theoretical foundations of TRR were derived, including the concept of the Maximum Activate Count (MAC) and the system's response once this limit is reached. The MAC specifies the maximum number of activations that a single row, denoted as a target row `TRn`, can sustain within a refresh period before adjacent rows, defined as victim rows, must be refreshed to prevent disturbance errors [51]. When the MAC limit is reached, the device either performs all required refresh commands before allowing further row activations or enters TRR mode, in which only the adjacent victim rows are refreshed [51].

An analysis of the relevant mode register configuration, as summarized in table 7, reveals that `MR0` is used for mode selection, controlling either RFM or or TRR operation. Specifically, TRR control applies only when `MR0 OP[2] = 0b`, whereas RFM is selected when `MR0 OP[2] = 1b`. The TRR mode itself is defined by `MR24 OP[7]`, where a value of `1` enables TRR mode and `0` disables it. The TRR mode BAn, which is indicating the bank of the target row, is specified by `MR24 OP[6:4]`. Furthermore, `MR24 OP[3]` determines the MAC type, with `0` corresponding to a limited MAC and `1` to an unlimited MAC. The MAC value is encoded in `MR24 OP[2:0]`, where `000b` represents an unlimited MAC, while any other configuration specifies a finite MAC value. Notably, when an unlimited MAC is configured by setting `MR24 OP[2:0] = 000b` and `MR24 OP[3] = 1`, TRR operation is not required, even if TRR mode itself is enabled via `MR24 OP[7]`. By default, TRR mode is disabled, as indicated by `MR24 OP[7] = 0b`. Despite this, the exact device behavior is vendor-specific.

Table 7: Mode Register Assignments

MR#	Function	OP7	OP6	OP5	OP4	OP3	OP2	OP1	OP0
0	Mode Select			Singleended mode			XORed RFM/TRR support	Latency mode	REF
24	TRR control	TRR mode	TRR mode BAn			UnItd MAC	MAC value		
24	RFM control	RAAMMT		RAAIMT					RFM

To confirm or refute whether TRR is implemented on the test devices, a deeper inspection of the relevant mode register entries is required, specifically `MR0 OP[2]`, `MR24 OP[3:0]`, and `MR24 OP[7]`.

Since mode registers can only be accessed through the memory controller, an indirect approach was considered. The Serial Presence Detect (SPD) mechanism was identified as a potential source of information, as it stores memory-related parameters such as MAC and possibly TRR configuration [83]. While DIMM-based systems typically include an SPD EEPROM, it is unclear whether such an EEPROM exists for soldered DRAM, as used in

Raspberry Pi devices. The proposed approach was therefore to access the system bus using I2C tools, dump the SPD EEPROM if present, and decode its contents according to the corresponding Double Data Rate (DDR) standard.

The workflow began with the initial detection of available I2C buses using `sudo i2cdetect -l`, which returned no entries, and `sudo i2cdetect -y 1`, which failed with an error indicating that `/dev/i2c-1` was not found. Consequently, the I2C interface was enabled via `sudo raspi-config` by selecting within the Interface Options the enabling I2C option. After reboot, it was assumed that the I2C driver would load during boot and that the `/dev/i2c-*` device files would become available.

Subsequent verification using `ls /dev/i2c*` confirmed the presence of three I2C buses: `/dev/i2c-1`, `/dev/i2c-13`, and `/dev/i2c-14`. Listing all I2C adapters with `sudo i2cdetect -l` showed one standard Synopsys DesignWare adapter and two SoC-integrated adapters. Unfortunately, a scan of the standard I2C bus using `sudo i2cdetect -y 1` did not detect any devices across the standard address range used by SPD EEPROMs (e.g., 0x50-0x57), as all addresses were reported as unused.

These results confirm that the I2C bus is active and operational, but no external I2C devices, including an SPD EEPROM at standard addresses, were detected. This observation is consistent with the fact that Raspberry Pi devices use soldered DRAM and aligns with previous findings reported in [4]. Consequently, `decode-dimms`, which is a primary x86 tool for reading memory device information from DIMM EEPROMs, could not be used. Furthermore, no interface exists for directly reading device information from the DRAM mode registers. Although it could theoretically be assumed that TRR is not enabled by default according to [51], this behavior is vendor-specific. Therefore, without access to non-public vendor documentation, the default TRR configuration of the test devices cannot be conclusively determined.

7.2 Bypass Techniques

Given the inability to conclusively determine whether TRR is implemented on the test devices, TRR must be assumed to be present. This assumption is further supported by the fact that previous Rowhammer tests failed to induce bitflips, even though the official Micron documentation suggests that TRR is disabled by default [51]. To address this, established TRR bypass techniques were considered, including Rowhammer fuzzers such as Blacksmith [22] and TRRespass [82].

Blacksmith [65] is a scalable Rowhammer fuzzer primarily designed for x86 systems. Introduced in 2022, it constructs novel non-uniform hammering patterns based on frequency,

phase, and amplitude to bypass modern in-DRAM TRR mitigations and successfully induce bitflips. TRRespass [68], introduced in 2020, is another Rowhammer fuzzer for x86 systems that automatically identifies TRR-bypassing access patterns. It generates a wide range of effective Rowhammer patterns on TRR-protected DRAM modules by varying two parameters, namely cardinality and location, and by employing non-conventional, many-sided Rowhammer patterns to circumvent both memory-controller-level and DRAM-level defenses. Within this work, the focus is placed on porting TRRespass, as the authors of [68] implemented a simplified ARMv8 version to test LPDDR4 and LPDDR4x memory and study the prevalence of the issue, although no public repository for this implementation exists.

Both Blacksmith and TRRespass rely on the availability of HugePages [77, 48, 27]. Initially, HugePages could not be used because the running kernel did not support HugeTLB or Transparent HugePages, as indicated by the absence of any output from `grep Huge /proc/meminfo`. An inspection of the kernel configuration using `/boot/config-$(uname -r)` revealed that while the architecture fundamentally supports HugePages, the kernel was compiled without enabling either Transparent HugePages or HugeTLBFS. As a result, HugePages were enabled by natively rebuilding the Linux kernel, as described in section 3.4. According to the Debian arm64 kernel documentation, HugeTLB page sizes of 2 MB and 1 GB are supported, but 1 GB HugeTLB pages must be pre-allocated at boot time via kernel command line arguments, as they cannot be allocated dynamically [27]. Only smaller page sizes, such as 64 KB, 2 MB, and 32 MB, can be allocated at runtime. Therefore, the kernel command line was extended to pre-allocate a single 1 GB HugeTLB page.

After enabling HugePages, we verified their availability. On the Raspberry Pi 4, the available sizes were 64 KB, 2 MB, 32 MB, and 1 GB, whereas on the Raspberry Pi 5 only 2 MB, 32 MB, and 1 GB HugePages were supported.

To ensure deterministic HugeTLB behavior, Transparent HugePages were explicitly disabled via a privileged write to `/sys/kernel/mm/transparent_hugepage/enabled`. Following kernel command line configuration and a system reboot, the kernel message buffer was filtered using `dmesg | grep -i huge` to verify which HugePage sizes were successfully pre-allocated. Finally, the active HugePage configuration was confirmed by querying `/sys/kernel/mm/hugepages/*/nr_hugepages`, which reports the number of HugePages currently in use for each supported size.

7.3 TRRespass ARM Port

This subsection focuses exclusively on the architectural adaptations required to port TRRespass from x86-based systems to the ARMv8 architecture in order to enable execution on the

ARM-based single-board computers used in this work, denoted as test devices.
The scope of this porting effort is deliberately constrained. The core fuzzing and hammering algorithms implemented in TRRespass remain unchanged, and no modifications are made to their fundamental logic or operation. Consequently, the algorithmic behavior and the effectiveness of the underlying attacks are not reassessed in this work, as these aspects are thoroughly analyzed and discussed in the original TRRespass publication [68].
Several porting challenges arise from the fundamental differences between x86 and ARMv8 architectures. A primary issue is the absence of x86-specific instructions and registers on ARM platforms, which necessitates the replacement of x86-only CPU instructions with ARMv8-compatible equivalents. This challenge is further compounded by the lack of direct counterparts for certain cache flush and timing instructions that are commonly available on x86 systems. In addition, compatibility with Linux hugepage support on ARM systems must be ensured, as outlined in the previous section. The migration also requires replacing x86-specific compiler flags with configurations suitable for ARMv8. To support heterogeneous ARM platforms across different test devices, system page sizes must be handled dynamically. Furthermore, hugepages need to be integrated to preserve contiguous memory regions, which are essential for both hammering and fuzzing operations.
As a result of these adaptations, the modified TRRespass tool executes successfully on the ARMv8-based platforms evaluated in [37]. Moreover, the ported TRRespass tool [38] establishes a foundation for future research on Rowhammer-related attacks and defenses in ARM-based systems.

8 Conclusion

The present study addresses the practical challenge of evaluating memory disturbance effects on contemporary Raspberry Pi platforms. Specifically, it focuses on implementing a Rowhammer-based proof-of-concept to investigate whether repeated memory accesses can induce observable bitflips on the Raspberry Pi 4 and 5. This work bridges the gap between theoretical vulnerability assessments and experimental verification on modern embedded ARM systems.

Regarding the degree of objective fulfillment, the proof-of-concept was successfully designed and implemented on both the Raspberry Pi 4 and 5 platforms. The implementation incorporates two practical cache-bypass techniques - eviction-based strategies and cache maintenance instructions, enabling uncached memory access for experimental hammering attempts. Furthermore, the DRAM mapping functionality was fully integrated using the ported DRAMA tool for ARM, allowing targeted memory row analysis on each test device. TRRespass was successfully ported but did not generate effective access patterns, demonstrating the current limitations of inducing Rowhammer bitflips on modern Raspberry Pi hardware.

While all core modules of the proof-of-concept are operational and verified, inherent hardware protections such as TRR, or page policy behavior prevent reproducible bitflips, limiting the observable impact of the implemented hammering techniques.

This work provides a fully operational and documented Rowhammer proof-of-concept specifically tailored to the Raspberry Pi 4 and 5 across multiple RAM configurations of 4 GB, 8 GB, and 16 GB. It establishes an ARM-adapted and publicly available combination of the DRAMA and TRRespass tools, thereby facilitating further research on embedded platforms. Moreover, the results confirm that, under standard operating conditions, modern Raspberry Pi devices exhibit resilience against conventional software-induced Rowhammer attacks, underscoring the effectiveness of contemporary embedded mitigation mechanisms.

Appendix A Listings

References

[1] 0x5ec1ab. *rowhammer_armv8*. 2019. URL: https://github.com/0x5ec1ab/rowhammer_armv8 (visited on 11/26/2025).

[2] AMD. *AMD's Zen CPU is now called Ryzen, and it might actually challenge Intel*. 2016. URL: https://arstechnica.com/gadgets/2016/12/amd-zen-performance-details-release-date/ (visited on 02/17/2026).

[3] AMD. *Developer Central*. 2020. URL: https://www.amd.com/en/developer.html (visited on 02/17/2026).

[4] Anandpreet Kaur, Pravin Srivastav, and Bibhas Ghoshal. "Flipping Bits Like a Pro". In: (2023). DOI: https://doi.org/10.1109/LES.2023.3298737. (Visited on 01/28/2026).

[5] Anandpreet Kaur, Pravin Srivastav, and Bibhas Ghoshal. "Work-in-Progress: DRAM-MaUT: DRAM Address Mapping Unveiling Tool for ARM Devices". In: (2022). DOI: https://doi.org/10.1109/CASES55004.2022.00027. (Visited on 11/26/2025).

[6] Andreas Kogler, Jonas Juffinger, Salman Qazi, Yoongu Kim, Moritz Lipp, Nicolas Boichat, Eric Shiu, Mattias Nissler, and Daniel Gruss. "Half-Double: Hammering From the Next Row Over". In: (2022). URL: https://www.usenix.org/system/files/sec22-kogler-half-double.pdf (visited on 11/26/2025).

[7] Andrew J. Walker, Sungkwon Lee, and Dafna Beery. "On DRAM Rowhammer and the Physics of Insecurity". In: (2021). DOI: https://doi.org/10.1109/TED.2021.3060362. (Visited on 11/26/2025).

[8] Antoine Plin, and Frédéric Fauberteau. "OpenGL GPU-Based Rowhammer Attack (Work in Progress)". In: (2025). DOI: https://doi.org/10.48550/arXiv.2509.19959. (Visited on 11/26/2025).

[9] Antoine Plin, Lorenzo Casalino, Thomas Rokicki, and Ruben Salvador. "Knock-Knock: Black-Box, Platform-Agnostic DRAM Address-Mapping Reverse Engineering". In: (2025). URL: https://arxiv.org/pdf/2509.19568 (visited on 11/26/2025).

[10] antpln. *Knock-Knock*. 2025. URL: https://github.com/antpln/Knock-Knock (visited on 11/26/2025).

[11] ARM. *ARM Architecture Reference Manual - ARMv8, for ARMv8-A architecture profil*. 2017. URL: https://developer.arm.com/documentation/ddi0487/latest (visited on 11/27/2025).

[12] ARM. *ARM Cortex-A72 MPCore Processor Technical Reference Manual r0p3*. URL: https://developer.arm.com/documentation/100095/0003/deb1353594352617?lang=en (visited on 11/26/2025).

[13] ARM. *Arm® DynamIQ™ Shared Unit*. 2019. URL: https://share.google/MaUeABdvCVJbcmSw2 (visited on 01/07/2026).

[14] ARM. *DC CIVAC, Data or unified Cache line Clean and Invalidate by VA to PoC*. URL: https://developer.arm.com/documentation/111107/2025-09/AArch64-Instructions/DC-CIVAC--Data-or-unified-Cache-line-Clean-and-Invalidate-by-VA-to-PoC?lang=en (visited on 01/28/2026).

[15] ARM. *DC CVAC, Data or unified Cache line Clean by VA to PoC*. URL: https://developer.arm.com/documentation/111180/2025-09_ASL1/AArch64-Instructions/DC-CVAC--Data-or-unified-Cache-line-Clean-by-VA-to-PoC (visited on 01/28/2026).

[16] ARM. *Introduction to the Cortex®-A76 core*. URL: https://developer.arm.com/documentation/100798/0401/Introduction-to-the-Cortex-A76-core?lang=en (visited on 11/26/2025).

[17] ARM. *SCTLR_EL1, System Control Register (EL1)*. URL: https://developer.arm.com/documentation/ddi0601/2025-09/AArch64-Registers/SCTLR-EL1--System-Control-Register--EL1- (visited on 11/26/2025).

[18] ARM. *System Control Register, EL1*. URL: https://developer.arm.com/documentation/ddi0488/h/system-control/aarch64-register-descriptions/system-control-register--el1 (visited on 11/26/2025).

[19] Arm Limited. *DCZID_EL0, Data Cache Zero ID Register*. 2024. URL: https://df.lth.se/~getz/ARM/SysReg/AArch64-dczid_el0.html (visited on 01/06/2026).

[20] Arm Limited. "PMCCNTR_EL0, Performance Monitors Cycle Counter". In: (2024). URL: https://df.lth.se/~getz/ARM/SysReg/pmu.pmccntr_el0.html (visited on 01/14/2026).

[21] Christian Helm, Soramichi Akiyama, and Kenjiro Taura. "Reliable Reverse Engineering of Intel DRAM Addressing Using Performance Counters". In: (2020). DOI: https://doi.ieeecomputersociety.org/10.1109/MASCOTS50786.2020.9285962. (Visited on 11/26/2025).

[22] comsec-group. *blacksmith*. 2024. URL: https://github.com/comsec-group/blacksmith (visited on 11/26/2025).

[23] Daniel Gruss. *Rowhammer Attacks: An Extended Walkthrough Guide*. 2017. URL: https://gruss.cc/files/sba.pdf (visited on 11/26/2025).

[24] Daniel Gruss, Martin Heckel, and Florian Adamsky. *Ten Years of Rowhammer: A Retrospect (and Path to the Future)*. 2024. URL: https://media.ccc.de/v/38c3-ten-years-of-rowhammer-a-retrospect-and-path-to-the-future#t=1177 (visited on 11/26/2025).

[25] Daniel Gruss, Moritz Lipp, Michael Schwarz, Daniel Genkin, Jonas Juffinger, Sioli O'Connell, Wolfgang Schoechl, and Yuval Yarom. "Another Flip in The Wall Of Rowhammer Defense". In: (2018). DOI: https://doi.org/10.48550/arXiv.1710.00551. (Visited on 11/26/2025).

[26] Dayeon Kim, Hyungdong Park, Inguk Yeo, Youn Kyu Lee, Youngmin Kim, Hyung-Min Lee, and Kon-Woo Kwon. "Rowhammer Attacks in Dynamic Random-Access Memory and Defense Methods". In: (2024). DOI: https://doi.org/10.3390/s24020592. (Visited on 11/26/2025).

[27] Debian. *Hugepages*. 2023. URL: https://wiki.debian.org/Hugepages (visited on 02/02/2026).

[28] Felix Cloutier. *CLFLUSH - Flush Cache Line*. URL: https://www.felixcloutier.com/x86/clflush (visited on 01/28/2026).

[29] Felix Cloutier. *RDTSC — Read Time-Stamp Counter*. 2023. URL: https://www.felixcloutier.com/x86/rdtsc (visited on 02/09/2026).

[30] Florian Adamsky. "Rowhammer on Raspberry Pi's: Assignment Sheet". In: (2025). (Visited on 02/04/2026).

[31] hammertux. *rowhammering*. 2019. URL: https://github.com/hammertux/rowhammering (visited on 11/26/2025).

[32] Intel. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. 2023. URL: https://www.intel.com/content/www/us/en/content-details/671488/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html (visited on 12/03/2025).

[33] isec-tugraz. *armageddon*. 2017. URL: https://github.com/isec-tugraz/armageddon (visited on 11/26/2025).

[34] isec-tugraz. *drama*. 2017. URL: https://github.com/isec-tugraz/drama (visited on 11/26/2025).

[35] isec-tugraz. *flipfloyd*. 2018. URL: https://github.com/isec-tugraz/flipfloyd (visited on 11/26/2025).

[36] isec-tugraz. *halfdouble*. 2022. URL: https://github.com/isec-tugraz/halfdouble (visited on 11/26/2025).

[37] Jakob Drescher. "Rowhammer on Raspberry Pi's: Bachelor Thesis". In: (2026). (Visited on 02/24/2026).

[38] Jakob Drescher. *Rowhammer on Raspberry Pi's: Github*. 2026. URL: https://github.com/averageTalking/rowhammer-raspberry (visited on 02/24/2026).

[39] japanninja74. *ES_Security_rowhammer_rpi3*. 2023. URL: https://github.com/japanninja74/ES_Security_rowhammer_rpi3 (visited on 11/26/2025).

[40] JEDEC. *JESD209-3, LPDDR3 Specification*. URL: %5Bhttps://www.jedec.org/sites/default/files/docs/JESD209-3.pdf%5D(https://www.jedec.org/sites/default/files/docs/JESD209-3.pdf) (visited on 01/29/2026).

[41] JEDEC. *JESD209-4, LPDDR4 Specification*. 2024. URL: https://www.jedec.org/standards-documents/docs/jesd209-4b (visited on 11/26/2025).

[42] JEDEC. *JESD209-4, LPDDR4X Specification*. 2025. URL: https://www.jedec.org/standards-documents/docs/jesd209-4-1 (visited on 11/26/2025).

[43] lulu89. *RPi3, RPi4 Hardware - lulu98/projects-with-sel4 GitHub Wiki*. 2022. URL: https://github-wiki-see.page/m/lulu98/projects-with-sel4/wiki/RPi3%2C-RPi4-Hardware (visited on 01/07/2026).

[44] Martin Heckel, and Florian Adamsky. "Flipper: Rowhammer on Steroids". In: (2025). DOI: https://doi.org/10.46586/uasc.2025.002. (Visited on 11/26/2025).

[45] Martin Heckel, and Florian Adamsky. "Reverse-Engineering Bank Addressing Functions on AMD CPUs". In: (2023). URL: https://dramsec.ethz.ch/papers/revengamd.pdf (visited on 01/15/2026).

[46] Martin Heckel, Florian Adamsky, Jonas Juffinger, Fabian Rauscher, and Daniel Gruss. "Verifying DRAM Addressing in Software". In: (2025). URL: https://forschung.hof-university.de/de/publikationen/1343-verifying-dram-addressing-in-software (visited on 02/17/2026).

[47] Matthew Blackmore. "A Quantitative Analysis of Memory Controller Page Policies". Masterthesis. Portland State University, 2013. URL: https://pdxscholar.library.pdx.edu/cgi/viewcontent.cgi?article=1659&context=open_access_etds (visited on 11/26/2025).

[48] mbechtel2. *MemoryAwareDOS*. 2021. URL: https://github.com/mbechtel2/MemoryAwareDOS (visited on 11/26/2025).

[49] Michael Bechtel, and Heechul Yun. "Memory-Aware Denial-of-Service Attacks on Shared Cache in Multicore Real-Time Systems". In: (2021). DOI: https://doi.org/10.1109/TC.2021.3108044. (Visited on 11/26/2025).

[50] Michael Schwarz. "DRAMA: Exploiting DRAM Buffers for Fun and Profit". Mastherthesis. Graz University of Technology, 2016. URL: https://blackhat.com/docs/eu-16/materials/eu-16-Schwarz-How-Your-DRAM-Becomes-A-Security-Problem-wp.pdf (visited on 01/28/2026).

[51] Micron. *LPDDR4X/LPDDR4 SDRAM*. URL: https://www.mouser.com/datasheet/2/671/z4bm_embedded_lpddr4x_lpddr4-3193428.pdf?srsltid=AfmBOoosiPjdpHJFHgsNPLezD5lZeKOVsm-CQFsx3nFMOKjtvkmCLpQ6 (visited on 11/26/2025).

[52] micron. *Micron FBGA and component marking decoder*. URL: https://www.micron.com/sales-support/design-tools/fbga-parts-decoder (visited on 01/23/2026).

[53] micron. *MT53E1G32D2FW-046 WT:B part detail*. URL: https://www.micron.com/products/memory/dram-components/lpddr4/part-catalog/part-detail/mt53e1g32d2fw-046-wt-b (visited on 11/26/2025).

[54] micron. *MT53E2G32D4DE-046 WT:C part detail*. URL: https://www.micron.com/products/memory/dram-components/lpddr4/part-catalog/part-detail/mt53e2g32d4de-046-wt-c (visited on 11/26/2025).

[55] micron. *MT53E2G32D4NQ-046 WT:A part detail*. URL: https://www.micron.com/products/memory/dram-components/lpddr4/part-catalog/part-detail/mt53e2g32d4nq-046-wt-a (visited on 11/26/2025).

[56] micron. *MT53E4G32D8CY-046 WT:C part detail*. URL: https://www.micron.com/products/memory/dram-components/lpddr4/part-catalog/part-detail/mt53e4g32d8cy-046-wt-c (visited on 11/26/2025).

[57] Moritz Lipp. "Cache Attacks and Rowhammer on ARM". Masterthesis. TU Graz, 2016. URL: https://mlq.me/download/master_thesis.pdf (visited on 11/26/2025).

[58] Moritz Lipp. "Exploiting Microarchitectural Optimizations from Software". PhD Thesis. TU Graz, 2021. URL: https://diglib.tugraz.at/download.php?id=61adc85670183&location=browse (visited on 02/13/2026).

[59] Moritz Lipp, Daniel Gruss, Raphael Spreitzer, Clémentine Maurice, and Stefan Mangard. “ARMageddon: Cache Attacks on Mobile Devices”. In: (2016). URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_lipp.pdf (visited on 11/26/2025).

[60] Moritz Lipp, Misiker Tadesse Aga, Michael Schwarz, Daniel Gruss, Clémentine Maurice, Lukas Raab, and Lukas Lamster. “Nethammer: Inducing Rowhammer Faults through Network Requests”. In: (2018). DOI: https://doi.org/10.48550/arXiv.1805.04956. (Visited on 11/26/2025).

[61] Niels Hagoort. *How is Virtual Memory Translated to Physical Memory?* 2020. URL: https://www.vmware.com/docs/how-is-virtual-memory-translated-to-physical-memory (visited on 02/09/2026).

[62] Niels Hagoort. *How is Virtual Memory Translated to Physical Memory?* 2020. URL: https://blogs.vmware.com/cloud-foundation/2020/03/03/how-is-virtual-memory-translated-to-physical-memory/ (visited on 11/26/2025).

[63] NordVPN. *Virtual address*. URL: https://nordvpn.com/de/cybersecurity/glossary/virtual-address/?srsltid=AfmBOor2uu2i2fjTsiQ9Fy-Sem10K6rX5hm2kaN893oCegcqyIR9hhEZ (visited on 02/09/2026).

[64] Onur Mutlu, and Jeremie S. Kim. “RowHammer: A Retrospective”. In: (2019). DOI: https://doi.org/10.48550/arXiv.1904.09724. (Visited on 11/26/2025).

[65] Patrick Jattke, Victor van der Veen, Pietro Frigo, Stijn Gunter, and Kaveh Razavi. “BLACKSMITH: Scalable Rowhammering in the Frequency Domain”. In: (2022). DOI: https://doi.org/10.1109/SP46214.2022.9833772. (Visited on 11/26/2025).

[66] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. “DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks”. In: (2016). URL: https://gruss.cc/files/drama.pdf (visited on 11/26/2025).

[67] Pietro Frigo, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “Grand Pwning Unit: Accelerating Microarchitectural Attacks with the GPU”. In: (2018). DOI: https://doi.org/10.1109/SP.2018.00022. (Visited on 01/28/2026).

[68] Pietro Frigo, Emanuele Vannacci, Hasan Hassan, Victor van der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “TRRespass: Exploiting the Many Sides of Target Row Refresh”. In: (2020). DOI: https://doi.org/10.1109/SP40000.2020.00090. (Visited on 11/26/2025).

[69] Raphael Spreitzer, and Thomas Plo. “Cache-Access Pattern Attack on Disaligned AES T-Tables”. In: (2013). DOI: 10.1007/978-3-642-40026-1_13. (Visited on 01/12/2026).

[70] Raspberry Pi. *linux*. 2026. URL: https://github.com/raspberrypi/linux (visited on 01/21/2026).

[71] Raspberry Pi. *Processors*. URL: https://www.raspberrypi.com/documentation/computers/processors.html (visited on 11/26/2025).

[72] RedHat. *Was ist der Unterschied zwischen ARM und x86?* 2022. URL: https://www.redhat.com/de/topics/linux/was-ist-der-unterschied-zwischen-arm-und-x86 (visited on 11/26/2025).

[73] Rui Qiao, and Mark Seaborn. “A New Approach for Rowhammer Attacks”. In: (2016). DOI: https://doi.org/10.1109/HST.2016.7495576. (Visited on 11/26/2025).

[74] The kernel development community. *Examining Process Page Tables*. URL: https://www.kernel.org/doc/html/latest/admin-guide/mm/pagemap.html (visited on 11/26/2025).

[75] The kernel development community. *Page Tables*. URL: https://docs.kernel.org/mm/page_tables.html (visited on 11/26/2025).

[76] thePi0neer. *How to determine the Pi 5 inclusivness*. 2025. URL: https://forums.raspberrypi.com/viewtopic.php?t=392384 (visited on 01/07/2026).

[77] user13938059. *HugePages on Raspberry Pi 4*. 2022. URL: https://stackoverflow.com/questions/64913818/hugepages-on-raspberry-pi-4/64914391#64914391 (visited on 01/21/2026).

[78] Victor van der Veen, Martina Lindorfer, and Yanick Fratantonio. “GuardION: Practical Mitigation of DMA-Based Rowhammer Attacks on ARM”. In: (2018). DOI: 10.1007/978-3-319-93411-2_5. (Visited on 01/28/2026).

[79] Victor van der Veen, Yanick Fratantonio, Martina Lindorfer, Daniel Gruss, Clémentine Maurice, Giovanni Vigna, Herbert Bos, Kaveh Razavi, and Cristiano Giuffrida. “Drammer: Deterministic Rowhammer Attacks on Mobile Platforms”. In: (2016). DOI: https://doi.org/10.1145/2976749.2978406. (Visited on 11/26/2025).

[80] vusec. *drammer*. 2016. URL: https://github.com/vusec/drammer (visited on 11/26/2025).

[81] vusec. *guardion*. 2018. URL: https://github.com/vusec/guardion (visited on 02/13/2026).

[82] vusec. *TRRespass*. 2021. URL: https://github.com/vusec/trrespass (visited on 11/26/2025).

[83] Wikipedia. *Serial presence detect*. 2026. URL: https://en.wikipedia.org/wiki/Serial_presence_detect (visited on 01/26/2026).

[84] Xianwei Zhang. *Virtual Memory*. 2015. URL: https://people.cs.pitt.edu/~xianeizhang/notes/vm.html (visited on 02/09/2026).

[85] Yichen Jiang, Huifeng Zhu, Haoqi Shan, Xiaolong Guo, Xuan Zhang, and Yier Jin. “TRRScope: Understanding Target Row Refresh Mechanism for Modern DDR Protection”. In: (2022). DOI: https://doi.org/10.1109/HOST49136.2021.9702274. (Visited on 11/26/2025).

[86] Yohannes Bekele, Ahmed Yiwere, and Daniel Limbrick. “Rowhammer Attacks on the Raspberry Pi 3B+”. In: (). URL: https://par.nsf.gov/servlets/purl/10220639 (visited on 11/26/2025).

[87] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, and Konrad Lai Onur Mutlu. “Flipping Bits in Memory Without Accessing Them”. In: (2014). DOI: https://doi.org/10.1145/2678373.2665726. (Visited on 11/26/2025).

[88] Yuan Xiao, Xiaokuan Zhang, Yinqian Zhang, and Radu Teodorescu. “One Bit Flips, One Cloud Flops: Cross-VM Row Hammer Attacks and Privilege Escalation”. In: (2016). URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_xiao.pdf (visited on 11/26/2025).

[89] Zhenkai Zhang, Zihao Zhan, Daniel Balasubramanian, Xenofon Koutsoukos, and Gabor Karsai. “Triggering Rowhammer Hardware Fault on ARM: A Revisit”. In: (2018). DOI: https://doi.org/10.1145/3266444.3266454. (Visited on 11/26/2025).